Path-graph free resolutions of bifiltered chain complexes

Ulrich Bauer, Tamal K. Dey, Michael Kerber, Florian Russold, Matthias Söls

arxiv:2512.08652; CC BY 4.0

Editorial scope. The path algorithm assembles simplex-wise staircase resolutions into a free chain complex using lifted boundary maps and chain homotopies; correctness requires the full source-specific mapping-cone quasi-isomorphism argument, not a routine matrix reduction. Complete author mathematical body and full appendices retained under own CC BY4.0; original implementation is not run or certified.

Allowed standard prerequisites (editorial).

- Bigraded modules, homological algebra, mapping cones and standard sparse matrix primitives. Original author context and bibliography retained. Source-specific lifts, path and cycle constructions, exactness and complexity proofs fully included.

1 Introduction

Motivation and problem statement. Multi-parameter persistence is a branch of topological data analysis where a data set (e.g., a point cloud) is filtered with respect to two or more parameters and the topological evolution of the data when changing the parameters is analyzed. In this context, the first step of a computational pipeline for two parameters typically consists of the computation of a bifiltration of simplicial complexes, that is, a family of simplicial complexes indexed by $\mathbb{N}^2$ that grows when increasing the parameters. We refer to the parameter set $\mathbb{N}^2$ as

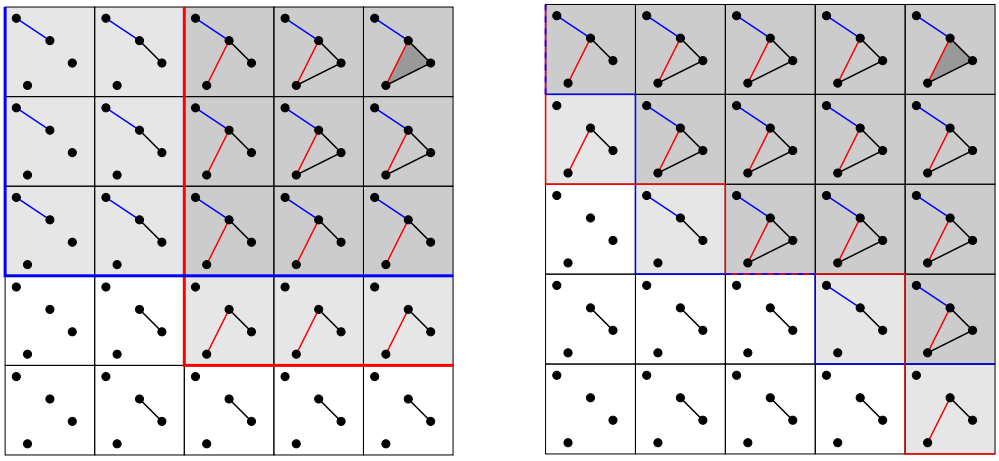


Figure 1: The bifiltration on the left is 1-critical while the bifiltration on the right is 3-critical. The support of every simplex is an upset, as visualized for two edges (red and blue, respectively).

the grades of the bifiltration. A bifiltration can be equivalently described by determining the support of each simplex, that is, the set of grades at which the simplex is part of the complex. A bifiltration is called free or 1-critical if the support of every simplex is a principal upset, that is, the upward closure of a single element in the parameter space. More generally, a bifiltration is k-critical if the support of a simplex is the upward closure of at most k elements; see Figure 1 for a visualization. A bifiltration gives rise to a bifiltered chain complex $C_\bullet$

$$0 \longleftarrow C_0 \xleftarrow{\partial_1} C_1 \longleftarrow \cdots \longleftarrow C_{d-1} \xleftarrow{\partial_d} C_d$$

with the k-simplices at a given grade forming the basis of the k-chains at that grade. Free and k-critical filtered chain complexes are defined analogously. Bifiltered chain complexes arise naturally in various constructions for topological data analysis, and several constructions produce bifiltrations that are not free. The most prominent example is given by the degree-Rips bifiltration [18, 22, 23, 6]. Other examples are the approximate multicover bifiltration [1] and Delaunay core bifiltration [5]. On the other hand, free bifiltrations are most suitable for algorithmic and computational purposes. There are fast algorithms for min-

imizing a bifiltered chain complex without changing its homology [13, 14, 21] as well as for computing the homology of a chain complex in terms of a minimal presentation [19, 17, 14, 3], but both approaches require free chain complexes as input. In homological algebra, a standard technique is to replace a general chain complex by a free one, connected to the original chain complex by a chain map that induces an isomorphism in homology (a quasi-isomorphism). The free chain complex together with the chain map is called a free resolution of the original complex. The problem studied in this paper is how to efficiently compute a free resolution of a non-free bifiltered chain complex. At the homology level, one way to address the case of a non-free bifiltration was proposed by Chacholski, Scolamiero and Vaccarino [7]: given a segment $C : C_{m+1} \to C_m \to C_{m-1}$ of a chain complex, the authors describe an algorithm to compute a free chain complex $F : F_{m+1} \to F_m \to F_{m-1}$ such that $H_m(C)$ and $H_m(F)$ are isomorphic, that is, a free implicit representation. This algorithm suffices if one is interested in a presentation of $H_m(C)$ for further processing. Methods to compute a projective implicit representation from families of simplicial complexes and general simplicial maps have been developed in [10, 11], where, as in our approach, graph theoretic methods are used to speed up computations. There are good reasons to work on the level of entire chain complexes instead. First of all, there are potential computational advantages, especially if one is interested in multiple homology dimensions (see Section 6). Moreover, the chain complex structure can encode subtle information on the data that is lost at the homology level: for example, two chain complexes may have isomorphic homology in every dimension without being quasi-isomorphic. Computing free resolutions of general chain complexes is a standard task in computational algebra, available in the computer algebra software Macaulay2 for a much wider range of chain complexes. In the context of applied topology, we are mostly interested in very large bifiltered chain complexes with millions of generators but a simple combinatorial structure (i.e., simplicial boundary maps). The goal of this paper is to develop specialized and highly optimized algorithms for this type of input data, which the general purpose algorithms implemented in existing computer algebra systems are not tailored for.

Contributions. Our main contribution is to propose two algorithms to compute a free resolution of a bifiltered chain complex. Both algorithms rely on the same simple idea of expanding any k-critical simplex in dimension p into a sequence of k free copies, with consecutive copies related by $(p+1)$-dimensional elements at the join of their grades. In algebraic terms, this corresponds to a free resolution of the upset module associated to the simplex. In order to construct a valid total complex, the algorithms introduce further maps to establish the chain complex property while maintaining quasi-isomorphism to the original filtered complex. Finding these maps is computationally inexpensive and takes place at a purely combinatorial level. The algorithms differ mainly in the choice of free resolution of the upset modules. In the first algorithm, the path algorithm, the free resolution corresponds to the chain complex of a filtered path graph (as a simple special case of a cellular resolution [4, 20]). We give an example of a family of simplicial bifiltrations so that the resulting free chain complex has a dense boundary matrix (for any choice of basis). This example shows tightness of the worst-case runtime $O(n^2)$, where n is the number of input generators (description size). The second algorithm, the log-path algorithm, extends the path algorithm by adding additional relations to the free resolution of a simplex, such that any pair of copies of a simplex is connected via a sequence of relations of logarithmic length. This ensures sparsity of the boundary matrices in the output chain complex, but also requires adding further higher relations (syzygies). Again, further maps are required to establish the chain complex property of the resulting total complex, allowing the algorithm to maintain sparsity and obtain a resolution with worst-case run time in $O(n \log^2 n)$. Our findings lead to an interesting observation: as shown by our worst case example, a minimal resolution may require dense boundary matrices, while a non-minimal resolution may actually admit a sparse matrix representation, with asymptotically fewer non-zero entries ($O(n \log^2 n)$ instead of $O(n^2)$, where n is the description size of the complex). This observation suggests that minimizing chain complexes does not necessarily speed-up subsequent algorithmic tasks, at least in certain worst-case examples. We provide implementations of the path and log-path algorithms, in addition to the Chacholski–Scolamiero–Vaccarino algorithm for computing free

implicit representations of homology. Systematic tests on various k-critical bifiltrations show that the overhead of the log-path algorithm over the path algorithm does not exceed a factor of 3 in run time and a factor of 2 in the number of non-zero entries for our examples, while showing the expected improvement on the mentioned worst-case examples. Subsequently minimizing the free chain complex yields a further significant reduction of the size. Furthermore, we consider the task of computing minimal presentations of homology in all degrees, comparing the approach of first computing a "global" free resolution with the approach of computing free implicit representations. Our results show a clear computational advantage for the global approach. Remarkably, for some instances, computing all minimal presentations using a free resolution is faster than computing a single minimal presentation using the Chacholski–Scolamiero–Vaccarino algorithm.

2 Bifiltered chain complexes

Bifiltrations. A simplicial bifiltration $\mathcal{K}$ is an abstract simplicial complex together with a collection of subcomplexes $(\mathcal{K}_s)_{s\in\mathbb{N}^2}$ such that $\mathcal{K}_s \subseteq \mathcal{K}_t$ whenever $s \leq t$ (which means that $s_1 \leq t_1$ and $s_2 \leq t_2$). As shown in Figure 1, each simplex σ enters $\mathcal{K}$ along a staircase bounding the support of σ, denoted $\operatorname{supp}(\sigma)$, which is an upset (an upward closed subset of $\mathbb{N}^2$). There is a unique minimal set of grades $\mathcal{G}(\sigma) := \{x_1, \ldots, x_m\} \subseteq \mathbb{N}^2$ such that $\operatorname{supp}(\sigma) = \{s \in \mathbb{N}^2 \mid \exists x_i \leq s\}$. If $m \leq k$ for each $\sigma \in \mathcal{K}$, then the bifiltration is k-critical.

Bipersistence modules. Simplicial bifiltrations give rise to bifiltered chain complexes. We first describe the elementary building blocks. Each p-simplex σ has a support $\operatorname{supp}(\sigma)$ with minimal generating set $\mathcal{G}(\sigma) = \{x_1, \ldots, x_m\}$, determining an upset module $U_{\{x_1,\ldots,x_m\}}$ (which we also denote by U_σ for brevity) given by

$$(U_\sigma)_s := \begin{cases} \Bbbk, & \text{if } \exists\, x_i \leq s, \\ 0 & \text{otherwise} \end{cases} \qquad \text{and} \qquad (U_\sigma)_{s,t} = \begin{cases} \mathrm{id}_\Bbbk, & \text{if } \exists\, x_i \leq s, \\ 0 & \text{otherwise} \end{cases}$$

as illustrated in Figure 2. This is a special case of a bipersistence module M, which is a family $(M_s)_{s\in\mathbb{N}^2}$ of vector spaces over a field

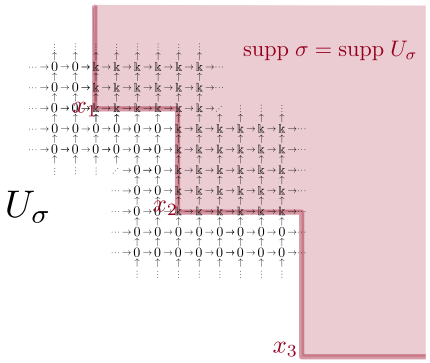


Figure 2: The upset module U_σ induced by the simplex σ.

$\Bbbk$ together with a family of homomorphisms $(M_{s,t} \colon M_s \to M_t)_{s \leq t \in \mathbb{N}^2}$. Although our results extend to arbitrary fields straightforwardly, we will stick to the case $\Bbbk = \mathbb{Z}_2$ in order to simplify the exposition. A morphism of bipersistence modules $\varphi : M \to N$ is a natural transformation, that is, a family of linear maps $(f_s)_{s \in \mathbb{N}^2}$ that commute with the structure maps of M and N: $f_t \circ M_{s,t} = N_{s,t} \circ f_s$ for $s \leq t$. A direct sum of upset modules, each generated by a single grade, is called free. A basis of a free bipersistence module F is a set of elements $\{b^{x_1}, \ldots, b^{x_n}\}$ where each $b^{x_i} \in F_{x_i}$, such that every $v \in F_y$, for any grade y, can be written in unique way as a linear combination of images of the basis elements under the structure maps (which are inclusions into the vector space $\bigcup_z F_z$). Note that the grades appearing in this linear combination with a non-zero coefficient must be less or equal to y. A morphism f between free modules is then determined by the images of the basis elements for the domain, and can be encoded by a matrix $[f]$.

Chain complexes. The upset modules associated to the simplices assemble to

$$0 \longleftarrow C_0 \xleftarrow{\partial_1} C_1 \xleftarrow{\partial_2} \cdots \xleftarrow{\partial_{i-1}} C_{i-1} \xleftarrow{\partial_i} C_i \xleftarrow{\partial_{i+1}} C_{i+1} \longleftarrow \cdots ,$$

which is a bifiltered chain complex with $C_i := \bigoplus_{\sigma \in \mathcal{K}^{(i)}} U_\sigma$ and boundary maps $\partial_i : C_i \to C_{i-1}$ inherited from the simplicial com-

plex $\mathcal{K}$, satisfying $\partial_i \circ \partial_{i+1} = 0$. The notion of a k-critical bifiltered chain complex is defined analogously to the setting of a simplicial bifiltration. Even though a bifiltered chain complex has a simple combinatorial structure, a free chain complex (a chain complex $F_\bullet$ with each F_i free) is preferred in computational (and algebraic) settings. This motivates the goal of finding a free resolution, that is, a free chain complex together with a chain map $F_\bullet \to C_\bullet$ that induces an isomorphism $H_i(F_\bullet) \cong H_i(C_\bullet)$ on homology for each $i \geq 0$. Such a chain map is called a quasi-isomorphism.

Data representation. The list of all simplices of a bifiltration $\mathcal{K}$, each with a lexicographically ordered list of the minimal generating set of its support, and a list of its facets gives a full combinatorial description of $\mathcal{K}$ and will serve as the input to our algorithms. This data representation generalizes to regular cell complexes, and in our case to bifiltered chain complexes where $\Bbbk = \mathbb{Z}_2$. For the simplicial case, the number of facets is constant for each simplex dimension, ensuring sparsity. Therefore the description size of the input is given, up to a constant factor, by the total number of generators in this list. Our output is a free resolution $F_\bullet$ of our input chain complex $C_\bullet$, again represented in the same data format.

3 Free implicit representations of homology

FI-reps. To compute the homology $H_m(C_\bullet)$ of a k-critical chain complex in a chosen dimension m, Chacholski, Scolamiero, and Vaccarino [7] provide a procedure to construct a free chain complex segment $X \xleftarrow{f} Y \xleftarrow{g} Z$ from the input segment

$$C_{m-1} \xleftarrow{\partial_m} C_m \xleftarrow{\partial_{m+1}} C_{m+1}. \tag{1}$$

such that $H_m(C_\bullet) \cong \ker f / \operatorname{im} g$. The pair of graded matrices $([f], [g])$ is called a free implicit representation (FI-rep) of $H_m(C_\bullet)$ [19], and it serves as input for the computation of minimal presentations of $H_m(C_\bullet)$ [19, 17].

The Chacholski–Scolamiero–Vaccarino algorithm. We consider an input complex $C_\bullet$ induced by a simplicial bifiltration

$\mathcal{K}$. The first step is to extend each U_σ in C_{m-1}, induced by an $(m-1)$-simplex σ, to a free upset module $U_{\{0\}}$. The resulting free module $D_{m-1} \coloneqq \bigoplus_{\sigma\in\mathcal{K}^{(m-1)}} U_{\{0\}}$, whose basis we denote by $\{b^0_\sigma\}_{\sigma\in\mathcal{K}^{(m-1)}}$, contains C_{m-1} as a submodule, since the support of U_σ is contained in the support of $U_{\{0\}}$. Thus, postcomposing ∂_m with this submodule inclusion does not affect the kernel: we have $\ker\partial_m = \ker\iota\circ\partial_m$. As a second step, we cover each U_σ in C_{m+1}, induced by an $(m+1)$-simplex σ by a free bipersistence module. This means that we replace each upset module U_σ by the free module $G_\sigma \coloneqq \bigoplus_{x_i\in\mathcal{G}(\sigma)} U_{\{x_i\}}$, connected to the upset module by a surjection $G_\sigma \twoheadrightarrow U_\sigma$ since the support of G_σ equals the support of U_σ. We call the basis elements $g^{x_i}_\sigma$ the generators of U_σ. The module G_{m+1} arises from C_{m+1} by covering U_σ by G_σ for each $(m+1)$-simplex via a canonical surjection $G_{m+1} \xrightarrow{\alpha_{m+1}} C_{m+1}$. Hence, precomposing ∂_{m+1} with α_{m+1} does not affect the image. The input (1) can now be replaced by (2) which has isomorphic homology.

$$D_{m-1} \xleftarrow{\iota\circ\partial_m} C_m \xleftarrow{\partial_{m+1}\circ\alpha_{m+1}} G_{m+1}, \tag{2}$$

As the third and final step it remains to replace C_m. We first construct G_m analogously to G_{m+1}, by substituting each U_σ in C_m by G_σ. Precomposing $\iota\circ\partial_m$ with the surjection $G_m \xrightarrow{\alpha_m} C_m$ yields a map $\gamma \coloneqq \iota\circ\partial_m\circ\alpha_m\colon G_m \to D_{m-1}$, as depicted in (3). For $\sigma\in\mathcal{K}^{(m)}$ with $\partial\sigma = \tau_0+\cdots+\tau_m$, the resulting map γ sends a generator g^x_σ in G_m to $b^0_{\tau_0}+\cdots+b^0_{\tau_m}$ in D_{m-1}.

$$\begin{array}{ccccccc}
 & & & & R_m & & \\
 & & & & \downarrow{\scriptstyle p^1_m} & & \\
 & & & \overset{\gamma}{\swarrow} & G_m & \xleftarrow{f^0_{m+1}} & G_{m+1} \\
 & & & & \downarrow{\scriptstyle \alpha_m} & & \downarrow{\scriptstyle \alpha_{m+1}} \\
D_{m-1} & \xleftarrow{\iota} & C_{m-1} & \xleftarrow{\partial_m} & C_m & \xleftarrow{\partial_{m+1}} & C_{m+1}
\end{array} \tag{3}$$

Replacing C_m by G_m makes it necessary to also replace the map $\partial_{m+1}\circ\alpha_{m+1}\colon G_{m+1}\to C_m$ by a map $f^0_{m+1}\colon G_{m+1}\to G_m$, representing ∂_{m+1} on the generators and also making the diagram commute. This map f^0_{m+1} sends each generator g^x_σ of an $(m+1)$-simplex σ with $\partial\sigma = \tau_0+\cdots+\tau_m$ to $f^0_{m+1}(g^x_\sigma) = g^{y_0}_{\tau_0}+\cdots+g^{y_m}_{\tau_m}$, where each $g^{y_i}_{\tau_i}$ is

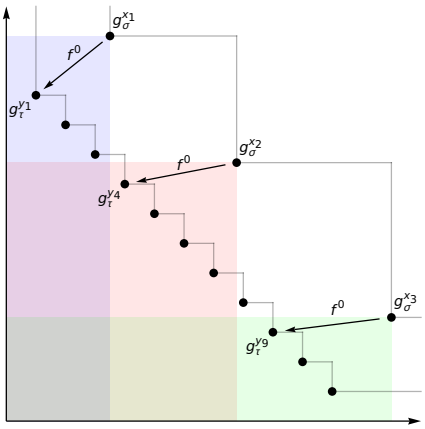


Figure 3: f^0_{m+1} sends each generator g^x_σ to a generator g^y_τ of the facet τ of σ whose grade y is in the downset of x (say, with the smallest first coordinate).

a generator of τ_i chosen such that its grade satisfies $y_i \leq x$; such a y_i always exists because the faces τ_i of a simplex σ are present in the bifiltration whenever σ itself is present. The map f^0_{m+1} thus makes the square in (3) commute and is therefore called a lift of ∂_{m+1}. See Figure 3 for an illustration. Note that the kernel of γ may not be isomorphic to the kernel of ∂_m; the surjection α_m maps each generator g^x_σ of an m-simplex in G_m to the same potential cycle in C_m, thus increasing the dimension of the kernel. This can be resolved by relating these generators appropriately. Assume that the generators $g^{x_1}_\sigma, \ldots, g^{x_k}_\sigma$ of U_σ are ordered w.r.t. the first coordinate of their grades. To represent U_σ correctly, two consecutive generators $g^{x_i}_\sigma$ and $g^{x_{i+1}}_\sigma$ have to be identified at the join $y_i = x_i \vee x_{i+1}$ of their grades by a relation $r^{y_i}_\sigma$. With the free bipersistence module $R_\sigma \coloneqq \bigoplus_{y_i} U_{\{y_i\}}$ with basis $\{r^{y_i}_\sigma\}_i$, we can present U_σ by generators and relations via

$$0 \longleftarrow U_\sigma \xleftarrow{\alpha_\sigma} G_\sigma \xleftarrow{p^1_\sigma} R_\sigma, \tag{4}$$

where $p^1_\sigma(r^{y_i}_\sigma) = g^{x_i}_\sigma + g^{x_{i+1}}_\sigma$. Since p^1_σ is injective, α_σ is surjective, and $\ker \alpha_\sigma = \operatorname{im} p^1_\sigma$, the sequence in (4) is a short exact sequence. This means that $(G_\sigma \xleftarrow{p^1_\sigma} R_\sigma, \alpha_\sigma)$ already determines a free resolution of U_σ, as defined in Section 2, where U_σ is considered as a chain

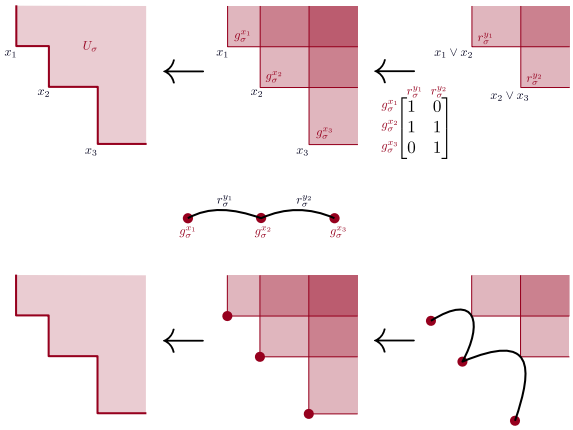


Figure 4: The top row illustrates the free resolution of U_σ as in Diagram 4 and the middle row shows its path resolution. We often visualize both at once as in the bottom row.

complex concentrated in degree 0. The free resolution in Diagram 4 has a simple combinatorial structure, given by a path graph $\mathcal{P}_\sigma$ with k vertices corresponding to the generators $g_\sigma^{x_i}$ and $k-1$ edges $\{g_\sigma^{x_i}, g_\sigma^{x_{i+1}}\}$ corresponding to the relations $r_\sigma^{y_i}$. Note that vertices and edges implicitly carry the grades of the generators and relations. The morphism p_σ^1 is represented by the (graded) incidence matrix of the graph. We call this free resolution the path resolution of U_σ, see Figure 4. Note that a path resolution is a special case of a cellular resolution [4, 20]. We now use the free resolution to replace the map $\partial_{m+1} \circ \alpha_{m+1}$ in (2), assembling the following output complex from (3)

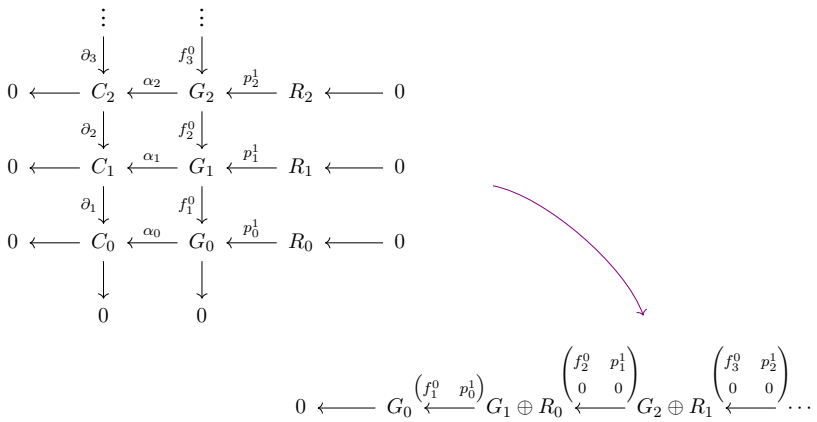
$$D_{m-1} \xleftarrow{\gamma_m} G_m \xleftarrow{\begin{pmatrix} f_{m+1}^0 & p_m^1 \end{pmatrix}} G_{m+1} \oplus R_m. \tag{5}$$

By construction, the homology of this complex segment in degree m is isomorphic to that of (2), and hence to that of the input complex.

4 Path algorithm

Description. We now turn to the problem of finding a free resolution, that is, a chain complex of free modules that is quasi-

isomorphic to the input $C_\bullet$, thus preserving homology in all dimensions. For this problem, extending the codomain of $\partial_i : C_i \to C_{i-1}$ to a free module generated in degree 0, as done in the Chacholski–Scolamiero–Vaccarino algorithm, is not feasible anymore, since this changes $H_{i-1}(C_\bullet)$. Instead, we now carry out the step of substituting C_i by its free resolution $G_i \xleftarrow{p_i^1} R_i \leftarrow 0 \leftarrow \cdots$ in every dimension i and replacing ∂_{i+1} by the (zeroth) lifts $f_{i+1}^0 \colon G_{i+1} \to G_i$, yielding maps $\begin{pmatrix} f_{i+1}^0 & p_i^1 \end{pmatrix}$ as illustrated in (6).



(6)

However, the sequence of maps, arising from this construction, does not yield a chain complex. Given a relation $r_\sigma^{y_j}$ in R_i, we obtain

$$f_i^0 \circ p_i^1(r_\sigma^{y_j}) = f_i^0(g_\sigma^{x_j}) + f_i^0(g_\sigma^{x_{j+1}}) = g_{\tau_0}^{z_1} + \cdots + g_{\tau_i}^{z_i} + g_{\tau_0}^{z'_1} + \cdots + g_{\tau_i}^{z'_i}.$$

This term (also the term $f_{i-1}^0 \circ f_i^0$) may not vanish, since the generators of σ may be sent to different generators of its boundary simplices τ_j by f_i^0, i.e., $g_{\tau_j}^{z_j} \neq g_{\tau_j}^{z'_j}$. However, it vanishes modulo relations, in the sense that its representative vanishes in $G_{i-1}/\operatorname{im} p_{i-1}^1$, see Lemma 1.

Lemma 1 (Path Lemma)**.** For any $g_\tau^{x_j}$, $g_\tau^{x_\ell}$ in G_{i-1}, there is a unique set of distinct relations $r_\tau^{w_j}, \ldots, r_\tau^{w_{\ell-1}}$ in R_{i-1} such that $p_{i-1}^1(r_\tau^{w_j} + \cdots + r_\tau^{w_{\ell-1}}) = g_\tau^{x_j} + g_\tau^{x_\ell}$.

Proof. This can be seen combinatorially. The free resolution of U_τ corresponds to the path graph $\mathcal{P}_\tau$, in the sense that the restriction of p_{i-1}^1 to a map $R_\tau \to G_\tau$ corresponds to the boundary map of the

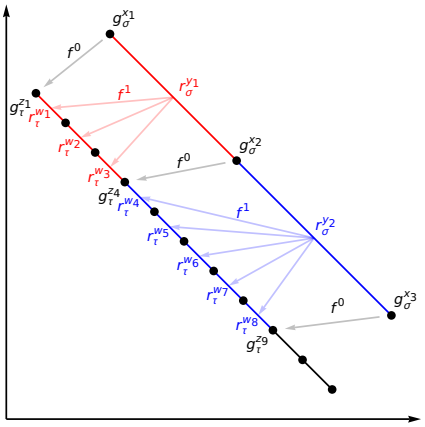
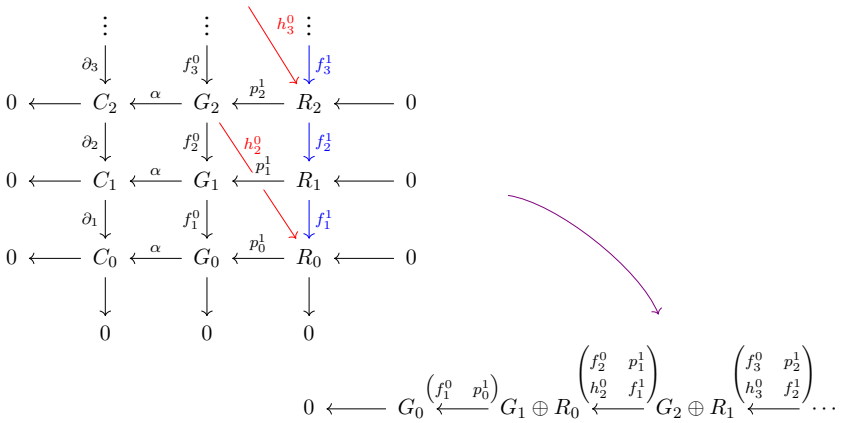


Figure 5: f_i^1 sends each edge r_σ^y to the path connecting the images of its endpoints under f_i^0.

path graph. The relations $r_\tau^{w_j}, \dots, r_\tau^{w_{\ell-1}}$ then define the unique path connecting the vertices $g_\tau^{x_j}$ and $g_\tau^{x_\ell}$ in $\mathcal{P}_\tau$. □

The map p_i^1 sends the edge $r_\sigma^{y_j}$ of $\mathcal{P}_\sigma$ to the vertices $g_\sigma^{x_j}$ and $g_\sigma^{x_{j+1}}$ which in turn are mapped to $g_{\tau_j}^{z_j}$ and $g_{\tau_j}^{z_j'}$ on $\mathcal{P}_{\tau_j}$ by f_i^0. We can therefore modify the boundary map in (6) by additionally sending $r_\sigma^{y_j}$ to the path $r_{\tau_j}^{w_1} + \dots + r_{\tau_j}^{w_\ell}$ guaranteed by Lemma 1, see Figure 5. This defines a correction term $f_i^1 : R_i \to R_{i-1}$ such that $p_{i-1}^1 \circ f_i^1 = f_i^0 \circ p_i^1$. The maps f_i^1 are called the first lifts of ∂_i. Even after adding the correction term f_i^1 to (6), the resulting maps may still fail to be boundary maps, since the composition $f_{i-1}^0 \circ f_i^0(g_\sigma^x)$ need not vanish for each generator g_σ^x in G_i, as seen in Figure 6. Again, $f_{i-1}^0 \circ f_i^0(g_\sigma^x)$ is a sum consisting of pairs $g_\tau^y, g_\tau^{y'}$ of possibly different generators of the same simplex τ, which can be connected by a path of relations as above. A second correction term $h_i^0 \colon G_i \to R_{i-2}$ is defined analogously to the lifts f_i^1. By construction, h_i^0 satisfies $p_{i-2}^1 \circ h_i^0 = f_{i-1}^0 \circ f_i^0$, making the following diagram commute:



(7)

Adding h_i^0 and f_i^1 to the boundary maps in (6) yields the desired free resolution of $C_\bullet$. The following Theorem, proven in Appendix A.1, concludes the correctness of the path algorithm.

Theorem 2. The chain complex on the right side of (7) is a free resolution of $C_\bullet$.

Path algorithm. For the description of the algorithm and its asymptotic time complexity, we assume that the input $C_\bullet$ is induced by a k-critical bifiltration of a simplicial complex with fixed dimension d, given as a list of simplices σ, each with its dimension, grades $\mathcal{G}(\sigma)$ and boundary $\partial\sigma$. The output is described by a list of generators and relations (each carrying a grade) as well as their images under the maps p_i^1, f_i^0, f_i^1 and h_i^0 (which together form the boundary maps of the free resolution). The output is initialized as an empty list.

1. Compute the bases of $G_i \oplus R_{i-1}$ and the matrices $[p_i^1]$. For each i-simplex σ and $x_j \in \mathcal{G}(\sigma)$, add a generator $g_\sigma^{x_j}$ to G_i. For any two consecutive $x_j, x_{j+1} \in \mathcal{G}(\sigma)$, add a relation $r_\sigma^{y_j}$ with $y_j = x_j \vee x_{j+1}$ to R_i. Moreover, define $p_i^1(r_\sigma^{y_j}) \coloneqq g_\sigma^{x_j} + g_\sigma^{x_{j+1}}$.

2. Compute the matrices $[f_i^0]$. For each generator g_σ^x in G_i, find generators $g_{\tau_0}^{x_0}, \dots, g_{\tau_i}^{x_i}$ for $\tau_0 + \cdots + \tau_i = \partial\sigma$ such that $x_j \leq x$ and define $f_i^0(g_\sigma^x) \coloneqq g_{\tau_0}^{x_0} + \cdots + g_{\tau_i}^{x_i}$.

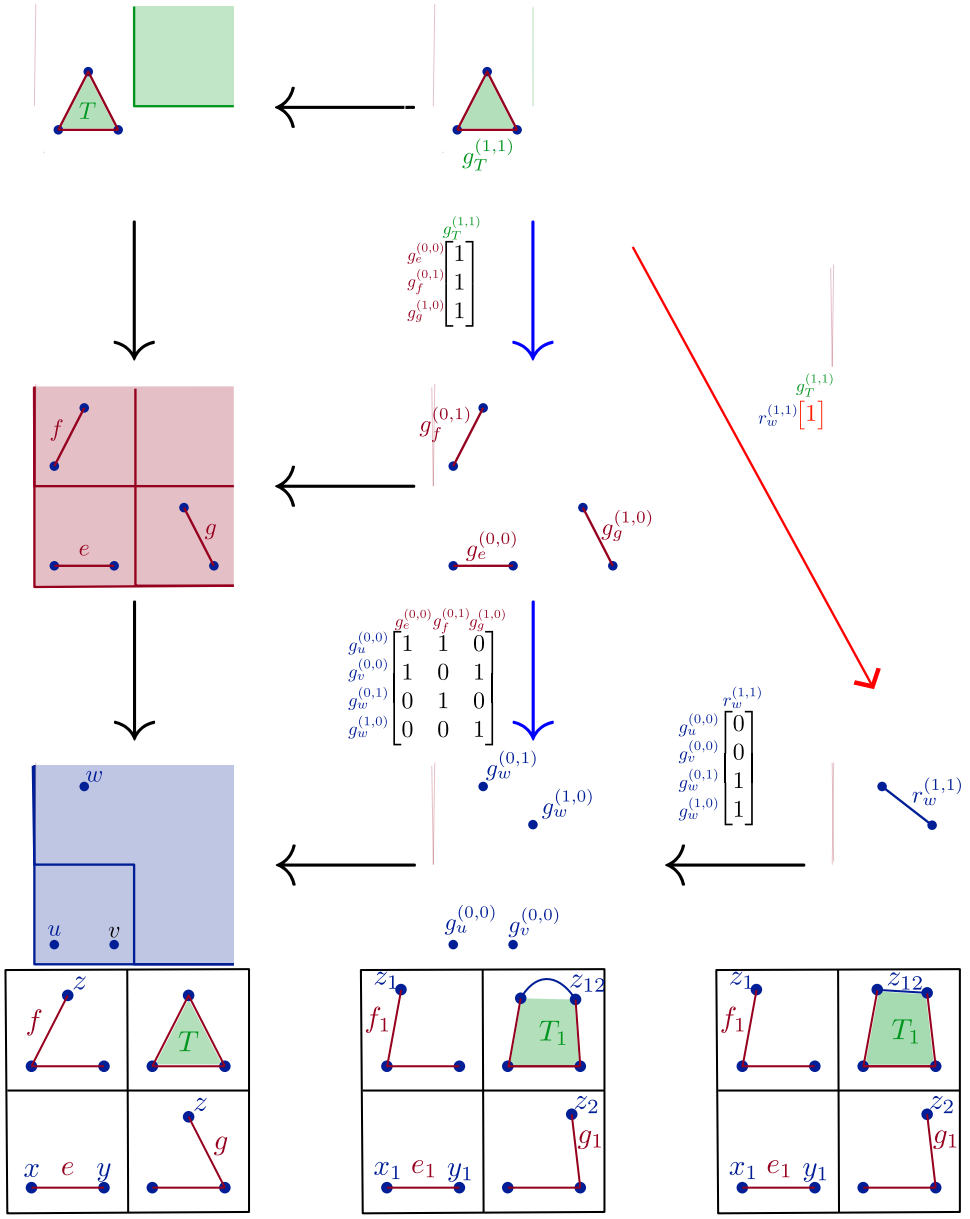


Figure 6: Construction of Diagram 7 for the bifiltration on the bottom left. The boundary defect $f_0^1 \circ f_1^1 \neq 0$, can be repaired by adding the red correction morphism which corresponds to a change of the boundary of T_1 in the bottom-middle cell complex. This yields the bottom-right output.

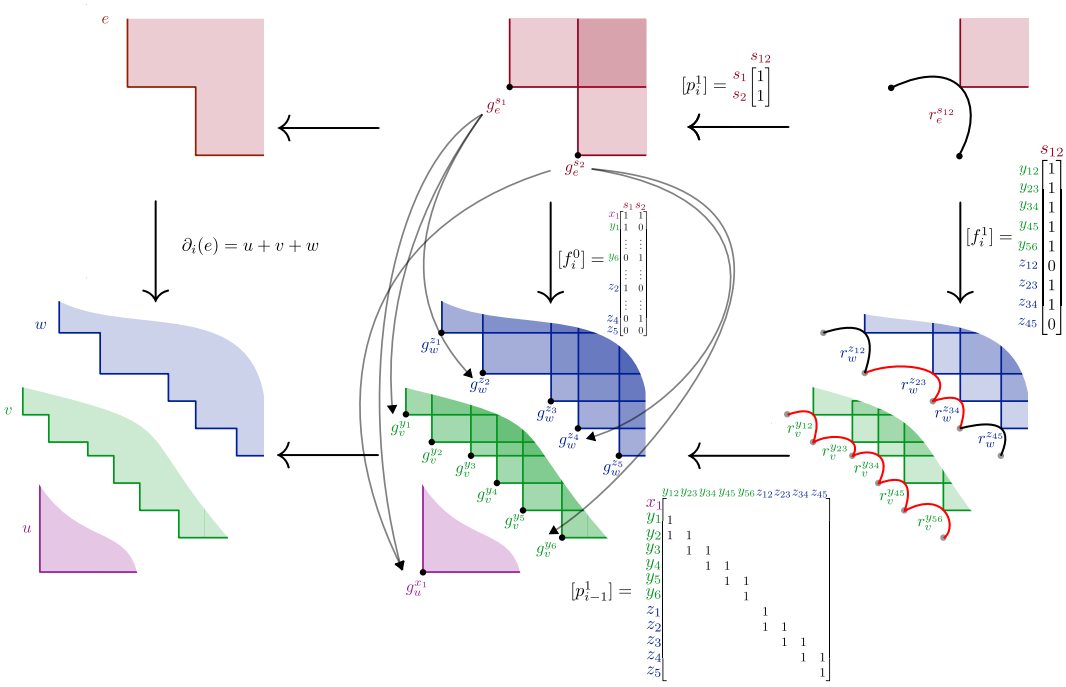


Figure 7: Construction of f_i^1. The relation $r_e^{s_{12}}$ is mapped to the paths $r_w^{z_{23}} + r_w^{z_{34}}$ and $r_v^{y_{12}} + \cdots + r_v^{y_{56}}$ in the path resolutions of w and v, highlighted in red.

3. Compute $[f_i^1]$ and $[h_i^0]$. For $[f_i^1]$, perform a matrix multiplication $[f_i^0][p_i^1]$. The resulting columns are indexed by the relations r_σ^y in R_i and consist of one pair of generators $g_\tau^{z_j}, g_\tau^{z_\ell}$ for each facet τ of σ. Such a pair is connected by a path $c_\tau := r_\tau^{w_j} + \cdots + r_\tau^{w_\ell}$ in $\mathcal{P}_\tau$, and we set $f_i^1(r_\sigma^y) := \sum_\tau c_\tau$; see Figure 7. For $[h_i^0]$, proceed analogously to multiply $[f_{i-1}^0][f_i^0]$. The resulting columns are indexed by the generators g_σ^x in G_i and consist of one pair of generators $g_\tau^{z_j}, g_\tau^{z_\ell}$ in G_{i-2} for each codimension 2 face τ of σ. Such a pair is connected by a path $c_\tau := r_\tau^{w_j} + \cdots + r_\tau^{w_\ell}$ in $\mathcal{P}_\tau$, and we set $h_i^0(g_\sigma^x) := \sum_\tau c_\tau$.

Complexity. The running time of the path algorithm is $O(nk)$ for a k-critical bifiltration of description size n; see Appendix A.2. This runtime is worst-case optimal, since the description size of a computed free resolution can be $\Omega(nk)$ in the worst-case. The

reason is that the paths in the computation of f_i^1 and h_i^0 can be of size up to k, and many simplices may require such long paths in the worst case. Figure 8 gives a construction for which the matrix $[h_2^0]$ is of size $\Omega(nk)$; see Figure 15 in the Appendix for a similar construction for $[f_1^1]$. A free chain complex is minimal if there is no quasi-isomorphic free chain complex with a smaller number of generators. In general, free resolutions computed by the path algorithm are not minimal. However, a slight modification of the construction in Figure 8 yields the following result (see Appendix A.3 for details).

Proposition 3. There is a k-critical simplicial bifiltration of description size n for which the path algorithm computes a free resolution represented by matrices of description size $\Omega(nk)$. This free resolution is minimal, and no other choice of basis yields a smaller description size.

5 Log-path algorithm

Shortcuts. In the preceding section, we leveraged the fact that the upset modules induced by simplices of a bifiltration admit path-shaped resolutions, allowing us to compute the correction terms f_i^1 and h_i^0 in a simple way. In the worst case, many generators and relations are mapped to long paths, making the matrices $[f_i^1]$ and $[h_i^0]$ dense ($\Omega(nk)$ size) and leading to a quadratic running time of $\Theta(nk)$ for the path algorithm. Such long paths can be avoided by introducing shortcuts in the path resolution $\mathcal{P}_\sigma$ (4), as illustrated in Figure 9. Recall that the grades $\mathcal{G}(\sigma) = \{x_1, \cdots, x_{n_\sigma}\}$ generating $\mathrm{supp}(\sigma)$ are totally ordered (by their first coordinate). Now any two vertices $g_\sigma^{x_j}$ and $g_\sigma^{x_\ell}$ with $j < \ell$ are connected by an additional shortcut edge if there are numbers $s \geq 0, t > 0$ with $j = s2^t$ and $\ell = (s+1)2^t$. Extending $\mathcal{P}_\sigma$ by this edge corresponds to adding a relation r_σ^z to R_σ, where $p_\sigma(r_\sigma^z) = g_\sigma^{x_j} + g_\sigma^{x_\ell}$ and $z = x_j \vee x_\ell$. In total, only $O(n_\sigma)$ many edges are added. This construction ensures that any two vertices $g_\sigma^{x_j}$ and $g_\sigma^{x_\ell}$ with $j < \ell$ are connected by a monotone path of length logarithmic in n_σ. A shortest monotone path can be constructed in a greedy way. We start in $g_\sigma^{x_j}$ and take the longest possible edge in each step that does not overshoot

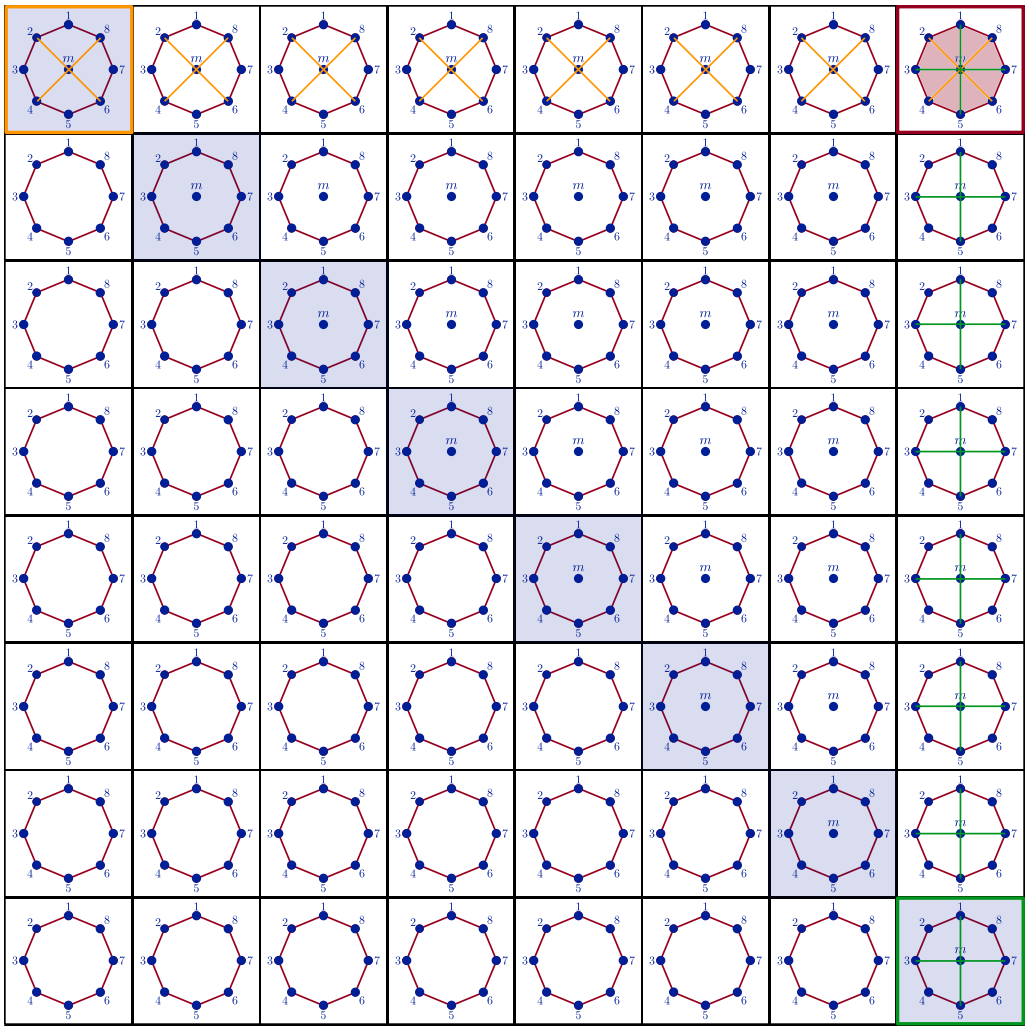


Figure 8: The wheel bifiltration consists of $\ell + 1$ 0-simplices, ℓ 1-simplices and ℓ 2-simplices, with $\ell = 8$ in this case. The only ℓ-critical 0-simplex is m entering the bifiltration along the blue staircase. This forces f_1^0 to map all generators of the orange (resp. green) 1-simplices to a generator of m with orange (resp. green) grade. The composition $f_1^0 \circ f_2^0$ then maps each generator of a purple 2-cell to the sum of a generator of m with orange grade and one with a green grade. The orange and green generators of m are connected by $\ell - 1$ relations. This yields $\ell - 1$ non-zero entries in each of the ℓ columns of $[h_i^0]$, making it dense.

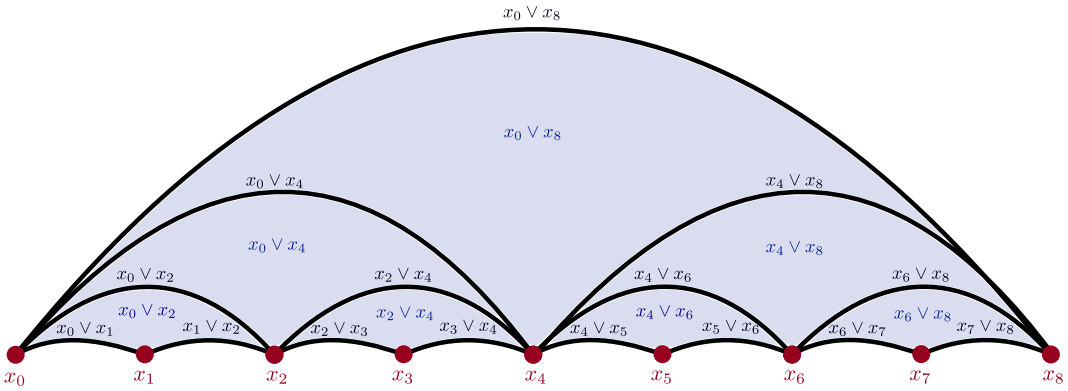


Figure 9: Log-path resolution labeled by the grades of its generators, relations and syzygies.

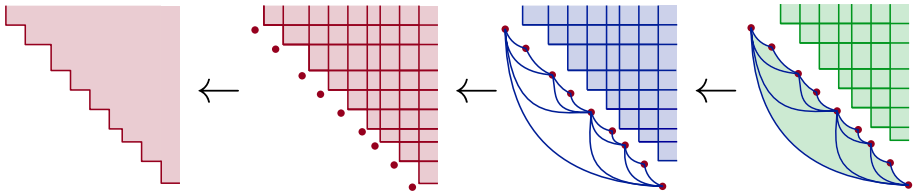


Figure 10: The new resolution of length 2 of an upset module U_σ.

$g_\sigma^{x_\ell}$. We conclude the following extension of Lemma 1 (proved in Appendix B.1).

Lemma 4 (Log-path Lemma)**.** For any $g_\sigma^{x_j}$, $g_\sigma^{x_\ell}$ in G_σ, there exist $r_\sigma^{y_1}, \dots, r_\sigma^{y_m}$ in R_σ with $m = O(\log n_\sigma)$ such that $p_\sigma^1(r_\sigma^{y_1} + \cdots + r_\sigma^{y_m}) = g_\sigma^{x_j} + g_\sigma^{x_\ell}$. Moreover, the elements $r_\sigma^{y_1}, \dots, r_\sigma^{y_m}$ can be computed in $O(\log n_\sigma)$ time.

Log-path resolutions. Lemma 4 appears to resolve the size issue: the same algorithm as in the previous section can be used, except that in the construction of h_i^0 and f_i^1 use Lemma 4 instead of Lemma 1. Then, the number of nonzero boundary coefficients for every generator and relation has only size $O(\log n_\sigma)$ instead of $O(n_\sigma)$, which would lead to an output boundary matrix with $O(n \log k)$ nonzero entries. Now recall that a key reason for why the path construction in (7) yields a chain complex is the uniqueness of paths between vertices. This leads to the vanishing of

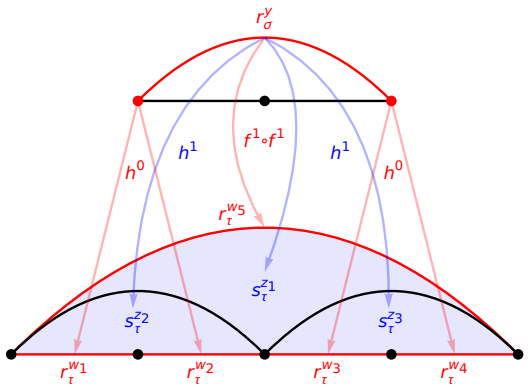


Figure 11: Cycle formed by paths $h^0 \circ p^1(r^y_\sigma)$ and $f^1 \circ f^1(r^y_\sigma)$ and its filling triangles $h^1(r^y_\sigma)$.

the terms $h^0_i \circ f^0_{i+1} + f^1_{i-1} \circ h^0_{i+1}$ and $h^0_i \circ p^1_i + f^1_{i-1} \circ f^1_i$ when composing boundary maps in (7). However, after adding shortcut edges the paths between vertices are no longer unique. As illustrated in Figure 11, for a relation r^y_σ, the terms $h^0_i \circ p^1_i(r^y_\sigma)$ and $f^1_{i-1} \circ f^1_i(r^y_\sigma)$, when restricted to a component, may correspond to different paths connecting the same endpoints and may therefore enclose a cycle. Any such cycle constitutes an obstruction to the vanishing of $h^0_i \circ p^1_i + f^1_{i-1} \circ f^1_i$. To eliminate these obstructions, we fill the cycles with faces, or, in algebraic terms, introduce syzygies, thus extending the modified relations to a free resolution, as depicted in Figure 10:

$$0 \longleftarrow U_\sigma \xleftarrow{p^0_\sigma} G_\sigma \xleftarrow{p^1_\sigma} R_\sigma \xleftarrow{p^2_\sigma} S_\sigma \longleftarrow 0. \tag{8}$$

The syzygies s^z_σ for S_σ correspond to the triangles with vertex grades $(x_{(s-1)2^t}, x_{s2^t}, x_{(s+1)2^t})$ for any odd $s > 0$; see Figure 9. The grade z is given by the grade of the longest edge, that is, $x_{(s-1)2^t} \vee x_{(s+1)2^t}$. The map p^2_σ is defined by sending s^z_σ to the sum of the three edges of the triangle. We call (8) the log-path resolution of U_σ, noting that it is a bifiltered two-dimensional simplicial chain complex $\mathcal{L}_\sigma$. Its construction yields the following property:

Lemma 5. Let $r^{y_1}_\sigma, \ldots, r^{y_l}_\sigma$ be a cycle in R_σ. Then there exists a unique chain $s^{z_1}_\sigma + \cdots + s^{z_q}_\sigma$ of $q \leq l-2$ elements in S_σ such that $p^2_\sigma(s^{z_1}_\sigma +$

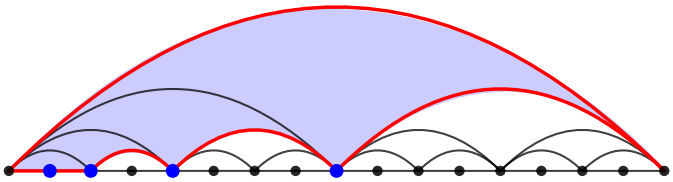


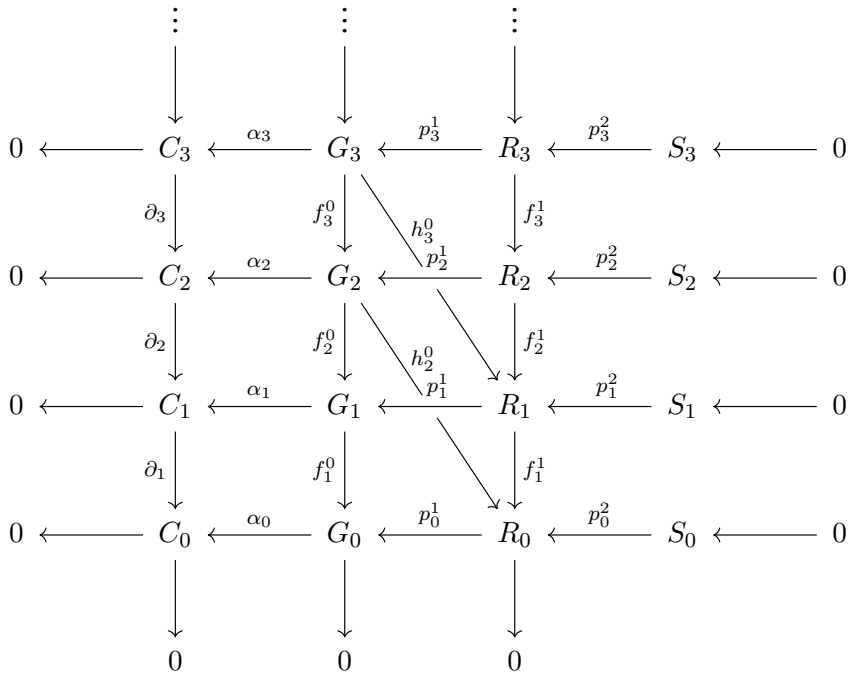
Figure 12: A simple red cycle in $\mathcal{L}$ for $k = 4$, together with the filling triangles in blue corresponding to the non-extremal vertices (with respect to the total order). Here each triangle $(x_{(s-1)2^t}, x_{s2^t}, x_{(s+1)2^t})$, with $s > 1$ odd, corresponds bijectively to its middle vertex x_{s2^t}.

$\cdots + s_\sigma^{z_q}) = r_\sigma^{y_1} + \cdots + r_\sigma^{y_l}$. Moreover, this chain can be computed in $O(l) = O(\log k)$ time.

To fill a cycle in $\mathcal{L}_\sigma$, we decompose it into simple cycles and use the observation that a simple cycle is filled with the triangles corresponding to the non-extremal vertices x_{s2^t} of the cycle, see Figure 12. See Appendix B.2 for details.

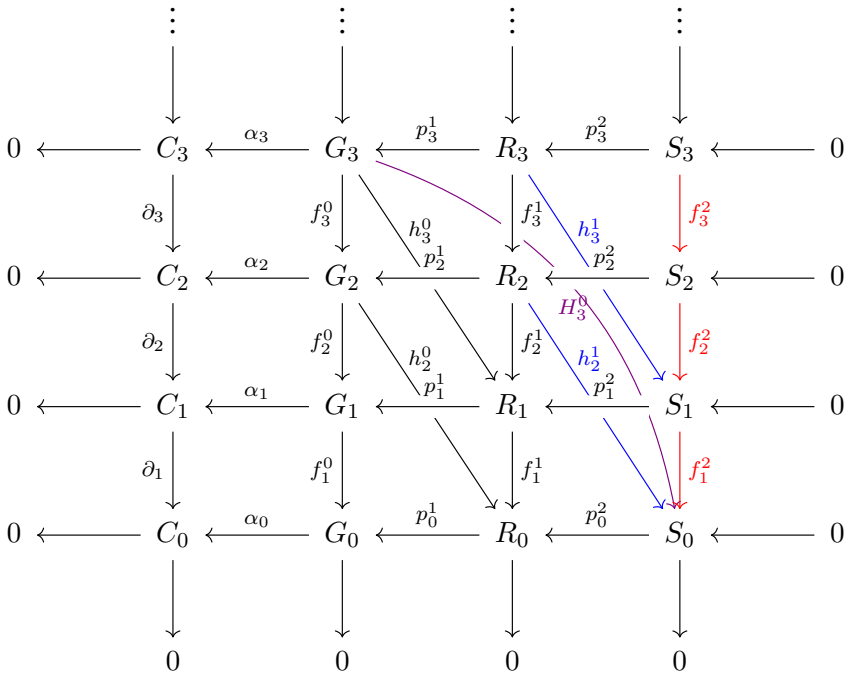
Log-path algorithm. We now describe the algorithm, again assuming that the input is a k-critical simplicial bifiltration of size n and constant dimension d with chain complex $C_\bullet$. Step 1: Compute the log-path resolution $\mathcal{L}_i : G_i \xleftarrow{p_i^1} R_i \xleftarrow{p_i^2} S_i$ of each module C_i. This requires iterating through the generators of each input simplex and adding a total of $O(n)$ relations and syzygies, which takes $O(n)$ time in total. Step 2: Compute $f_i^0 : G_i \to G_{i-1}$ in $O(n \log k)$ time as in Section 4. Step 3: Compute the maps $f_i^1 : R_i \to R_{i-1}$ and $h_i^0 : G_i \to R_{i-2}$ as in the path algorithm, except that paths between $g_\sigma^{x_j}, g_\sigma^{x_\ell}$ coming from the same simplex σ are computed via Lemma 4. We can find every path in $O(\log k)$ time and compute matrices $[f_i^1]$ and $[h_i^0]$ with at most $O(\log k)$ entries per column in a total running time of $O(n \log k)$ for this step. Step 4. All maps

computed in the previous steps yield the following extension of (7).



(9)

A construction as in Section 4 leads to boundary morphisms whose composition is non-zero. Again, the boundary defect is repaired by introducing correction maps f_i^2, h_i^1, and H_i^0:



(10)

Commutativity of Diagram (9) yields the following observation

Lemma 6. The following morphisms map to the kernel of p_i^1:

$$f_{i+1}^1 \circ p_{i+1}^2, \quad f_{i+1}^1 \circ f_{i+2}^1 + h_{i+2}^0 \circ p_{i+2}^1, \quad \text{and} \quad h_{i+2}^0 \circ f_{i+3}^0 + f_{i+1}^1 \circ h_{i+3}^0. \tag{11}$$

The log-path resolution $\mathcal{L}_i$ of C_i is the direct sum of the simplicial chain complexes $\mathcal{L}_\sigma$, taken over each simplex σ in C_i. The map p_i^1 is given by the 1-dimensional boundary map of this complex, and hence its kernel is the collection of cycles in the 1-skeletons of $\mathcal{L}_\sigma$. More specifically, each triangle s_σ^z in S_{i+1} is sent by p_{i+1}^2 to its boundary, a sum of three edges. By Lemma 6, applying f_{i+1}^1 to this boundary yields another cycle in R_i, which by construction of $\mathcal{L}_i$ decomposes as $\sum_\tau c_\tau$ for τ running over the facets of σ. For each such cycle c_τ, by Lemma 5 there is a unique chain b_τ in S_i such that $p_i^2(b_\tau) = c_\tau$, as illustrated in Figure 13. The map f_{i+1}^2 is then defined via $f_{i+1}^2(s_\sigma^z) := \sum_\tau b_\tau$, with τ running over the facets of σ. Thus f_{i+1}^2 repairs the boundary defect $f_{i+1}^1 \circ p_{i+1}^2$ as $f_{i+1}^1 \circ p_{i+1}^2 = p_i^2 \circ f_{i+1}^2$. Similar arguments lead to correction terms h_i^1, and H_i^0 satisfying

$$h_i^0 \circ p_i^1 + f_{i-1}^1 \circ f_i^1 = p_{i-2}^2 \circ h_i^1 \quad \text{and} \quad h_{i-1}^0 \circ f_i^0 + f_{i-2}^1 \circ h_i^0 = p_{i-3}^2 \circ H_i^0. \tag{12}$$

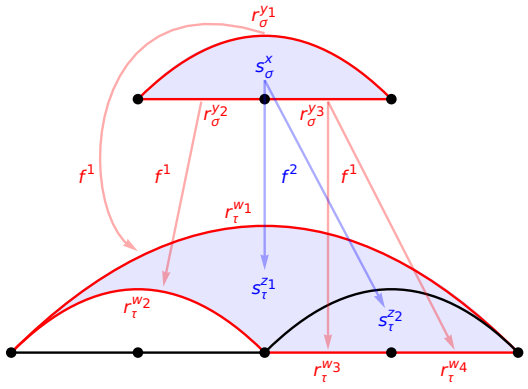


Figure 13: Cycle $f^1 \circ p^2(s_\sigma^x)$ and its filling triangles $f^2(s_\sigma^x)$.

Their construction is analogous to the above, and we omit the details. Note that in each of the three cases f_{i+1}^2, h_i^1, and H_i^0, the construction involves matrix multiplication (evaluation of the left hand sides in (12)), but also cycle filling (invoking Lemma 5). The former is efficient, as paths of logarithmic length lead to sparse matrices $[h_i^0]$ and $[f_i^1]$. More precisely, the column sparsity of $[f_i^1]$ and $[h_i^0]$ is $O(\log k)$, while in $[p_i^2]$ every column has exactly three non-zero entries. All sparse matrix products in Step 4 can thus be computed in $O(n \log^2 k)$ and the worst case column sparsity of the results is $O(\log^2 k)$. The resulting matrices contain cycles of length $O(\log^2 k)$, which can be filled by triangles in $O(\log^2 k)$ time. Overall, Step 4 takes $O(n \log^2 k)$ time. We can now assemble the output chain complex as

$$0 \longleftarrow G_0 \xleftarrow{\begin{pmatrix} f_1^0 & p_0^1 \end{pmatrix}} G_1 \oplus R_0 \xleftarrow{\begin{pmatrix} f_2^0 & p_1^1 & 0 \\ h_2^0 & f_1^1 & p_0^2 \end{pmatrix}} G_2 \oplus R_1 \oplus S_0 \xleftarrow{\begin{pmatrix} f_3^0 & p_2^1 & 0 \\ h_3^0 & f_2^1 & p_1^2 \\ H_3^0 & h_2^1 & f_1^2 \end{pmatrix}} G_3 \oplus R_2 \oplus S_1 \longleftarrow \cdots \tag{13}$$

Theorem 7. The chain complex (13) is a free resolution of $C_\bullet$.

For a proof, see Appendix B.3 and B.4. We summarize our observations on the running time in the following theorem.

Theorem 8. A free resolution of the chain complex $C_\bullet$ induced by a k-critical bifiltration of description size n can be computed in $O(n \log^2 k)$ time and has $O(n \log^2 k)$ size.

Applying this result to the modified wheel example (Appendix A.3) yields a non-minimal free resolution with smaller description size than any minimal one. Thus, perhaps surprisingly, minimal free resolutions are not asymptotically optimal in terms of description size.

6 Experimental evaluation

We implemented the path and log-path algorithm for computing a free resolution of a bifiltered chain complex in a C++ library called multi-critical [1]. The benchmark files are available upon request. The code expects a bifiltration given in scc2020 format [16], with the difference that for every simplex an arbitrarily long sequence of bigrades can be specified. The output is a chain complex in "proper" scc2020 format, representing a free filtered chain complex. Instead of outputting the result of our algorithms directly, the software can also post-process the filtered chain complex with the multi-chunk [15] library for minimizing the chain complex [13, 14]. All experiments were performed on a workstation with an Intel(R) Xeon(R) CPU E5-1650 v3 CPU (6 cores, 12 threads, 3.5GHz) and 64 GB RAM, running Ubuntu 16.04.5.

Test instances. We ran our experiments on a total of 342 test instances from different sources. First, we generated degree-Rips bifiltrations for (noisy) point samples drawn from a torus embedded in $\mathbb{R}^5$, from the "swiss roll" embedded in $\mathbb{R}^3$, and from the 7-dimensional unit cube, using the python package tadasets. In all cases, we generated the complex up to 3-simplices. Furthermore, we used a simple method for generating bifiltrations of a simplicial complex equipped with two non-negative real-valued functions f_0 and f_1 on its simplices, described in Figure 14. We employed this construction on meshes from the Aim@Shape

[1] `https://bitbucket.org/mkerber/multi_critical/src/main/`

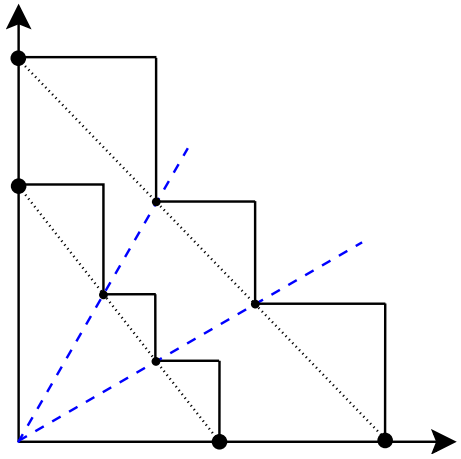


Figure 14: The figure shows the construction for two simplices σ and τ. The bigrades $(0, f_0(\sigma))$ and $(f_1(\sigma), 0)$ are connected by a line segment (dotted line), and the positive quadrant is equally dissected using $(k-2)$ lines through the origin (blue dashed lines, here $k = 4$). The intersection of dotted and dashed lines yield $(k-2)$ further bigrades for σ. The staircase for σ is drawn in solid. The same construction is done for τ; note that if $f_0(\sigma) \leq f_0(\tau)$ and $f_1(\sigma) \leq f_1(\tau)$, the staircase for σ is below the staircase for τ. In particular, the construction bifilters a simplicial complex if the functions f_0 and f_1 respect its face relations.

repository[2], using the squared mean curvature and the distance to the barycenter as the two functions. Furthermore, we bifiltered the Delaunay triangulations from the same point samples as above (torus, cube, swiss roll) using the minimum enclosing radius of a simplex and the average distance to the $\lfloor \log n \rfloor$-nearest neighbors as filter functions. In all cases, we generated k-critical instances with $k = 2, 4, 8$. We computed the filtration values with CGAL [9, 26, 12]. Finally, we generated wheel bifiltrations as in Figure 8 for different sample sizes.

Comparison of path and log-path algorithms. Table 1 shows some of the results obtained from both algorithms on these datasets; the results for the other instances are similar. In al-

[2] `http://visionair.ge.imati.cnr.it/`

most all cases, the log-path algorithm is slower than the path algorithm by a factor of up to 3 and produces an output complex of around twice the size. Also, it uses around twice as much memory (not shown in the table). Moreover, minimizing the chain complex (rightmost column of Table 1) further reduces the complex size significantly, generally at a small cost – sometimes it even saves time because the time used for minimization is less than the cost of producing the larger output file. The only exception is the wheel bifiltration, where our experiments show the expected asymptotic worst-case behavior, with the log-path algorithm outperforming the path algorithm. In this case, minimizing the chain complex produced by the log-path algorithm will necessarily introduce a dense matrix and destroy the advantage of the log-path algorithm – for instance, its running time in the instance in the last row of Table 1 is 33 seconds, with 14.4 seconds for minimization itself and 18 seconds to write the 2.3 GB output file.

			log-path		path		path+multi_chunk	
Type	#points	size	time	size	time	size	time	size
deg-Rips torus in $\mathbb{R}^5$	32	10MB	6.59	84MB	2.44	42MB	2.33	18MB
	36	18MB	12.3	156MB	4.52	77MB	4.34	33MB
	42	37MB	27.0	355MB	9.73	171MB	9.52	73MB
	48	71MB	54.2	725MB	19.1	343MB	19.2	155MB
Delaunay swiss roll in $\mathbb{R}^3$ criticality 4	5000	257MB	33.4	521MB	19.7	389MB	16.3	32MB
	10000	534MB	69.8	1.1GB	40.4	823GB	35.0	63MB
	20000	1.1GB	142	2.2GB	83.7	1.7GB	78.7	131MB
	40000	2.2GB	297	4.7GB	175	3.5GB	191	282MB
Wheel	2500	184KB	0.06	440KB	0.78	31MB	0.78	30MB
	5000	376KB	0.13	932KB	2.91	132MB	3.39	120MB
	10000	804KB	0.26	2.0MB	13.5	574MB	11.7	526MB
	20000	1.7MB	0.61	4.0MB	49.4	2.3GB	53.1	2.3GB

Table 1: Comparison of the path and the log-path algorithm. All times are in seconds. The "size" columns give the size of the input and output files. For deg-Rips and Delaunay, the numbers are averaged over 5 independently generated instances.

Computing minimal presentations. For a multi-critical filtered chain complex, we investigate the task of computing minimal presentation matrices for homology. We provide two variants: In the first variant, we compute a free resolution of the input chain complex with the path algorithm (which is usually

faster than the log-path algorithm, as seen in the previous experiment) and subsequently minimize using multi-chunk. We then compute minimal presentations for each homology degree separately using the mpfree library [17, 14]. The second variant operates by directly structuring the input complex into chain complex segments per degree. It then uses the Chacholski–Scolamiero–Vaccarino algorithm to compute a free implicit representation for each segment, before finally employing mpfree once more to generate a minimal presentation. The results of this experiment are presented in Table 2. For the degree-Rips instances, the first variant provides a modest improvement over the second. The reason is that the vast majority of simplices are in the top dimension 3, so that computing the presentation for H_2 is the bottleneck in the computation. This step, however, does not differ significantly in both approaches: most of the time is spent to determine which 3-simplices are killing 2-cycles, by reducing the boundary matrix for 2- and 3-simplices. For the bifunction instances, the speed-up of the first variant using free resolutions is much more pronounced. The reason is that these instances have a more balanced distribution of simplices over different dimensions: while for the second variant, the algorithm for H_k still has to find the bounding $(k+1)$-simplices for every k, the multi-chunk algorithm makes use of the clearing optimization [8] and hence avoids the reduction of large parts of the boundary matrices. Remarkably, this technique is so effective that computing all minimal presentations via free resolutions is faster than computing a single minimal presentation via the approach using the Chacholski–Scolamiero-Vaccarino algorithm, even though restricting to a single dimension k allows this approach to disregard all chains in dimensions other than $k+1$, k, or $k-1$.

Comparison with Macaulay2. While the Macaulay2 software includes a general `freeResolution` routine [25], it is unsuitable for our purposes. Its use requires converting our bigraded input into a graded chain complex over $\mathbb{Z}_2[x, y]$, a conversion that we found to be slow itself. More importantly, the core computation in Macaulay2 was orders of magnitude slower than our method, even on much smaller instances. We infer that the software was not designed for the large inputs typical of TDA. The

Type	#points	size	with path	with csv
deg-Rips cube in $\mathbb{R}^7$	32	8.7MB	5.66	6.22
	36	17MB	11.5	12.8
	42	35MB	28.7	34.0
	48	69MB	64.0	69.6
Delaunay torus in $\mathbb{R}^5$ criticality 8	140	129MB	14.7	79.7
	220	244MB	32.0	196
	340	435MB	56.8	403
	540	900MB	110	1017
hand, $k = 4$	34K	26MB	5.46	23.7
hand, $k = 8$	34K	55MB	13.1	55.3
hand, $k = 16$	34K	114MB	30.6	146
donna, $k = 2$	$503K$	170MB	44.9	131
donna, $k = 4$	503K	411MB	119	312
donna, $k = 8$	503K	895MB	282	860
pensatore, $k = 2$	998K	339MB	109	9585
pensatore, $k = 4$	998K	818MB	330	14054
pensatore, $k = 8$	998K	1.8GB	958	>4h

Table 2: Computing all minimal presentations with the path algorithm and with the Chacholski–Scolamiero–Vaccarino algorithm. All running times are in seconds.

conversion script is available on request.

7 Discussion

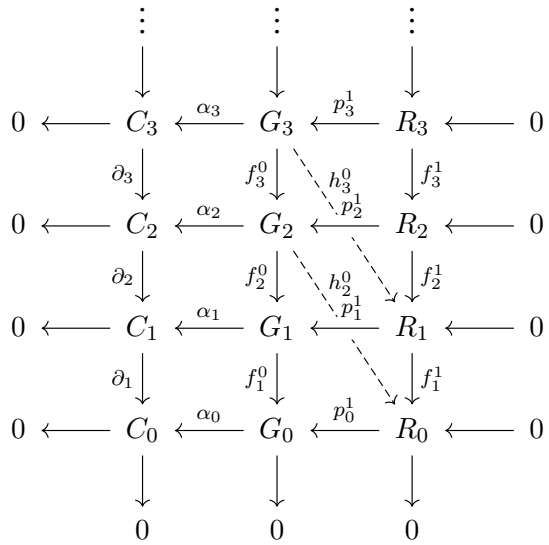
Our experimental evaluation suggests that the path algorithm often exhibits slightly better performance, which the log-path algorithm is more robust towards "bad" instances, with a relatively low overhead. Together, both variants contribute towards an efficient computational pipeline for multi-critical bifiltrations. Our results also complement recent development for computing degree-Rips bifiltrations more efficiently [2]. The finding that minimal chain complexes may necessitate a quadratic size in sparse matrix representations suggests that these representations may not be universally ideal for boundary matrices. Specifically, the matrix in Figure 8 admits a linear-size description, illustrating a gap in current approaches. It would be interesting to balance effective worst-case compression with efficient matrix processing, possibly by using an alternative data structure or by

selective shortcutting. Finally, our approach partially extends to simplicial complexes filtered with three parameters: a simplexwise minimal free resolution now has length 2, and suitable connecting maps compose into the same diagram as (10). Moreover, as described by Miller and Sturmfels [20], the simplexwise free resolution carries the structure of a planar graph, leading to a cubic-time algorithm. To break the cubic barrier, we will need to generalize the shortcut idea of Section 5 from paths to planar graphs, extending the free resolution of a simplex to length 3 (and introducing even more maps). We leave the details to future work.

A Proofs and details for the path algorithm

A.1 Proof of quasi-isomorphism

For convenience, recall that Diagram (7) consists of the construction



(14)

whose spaces and morphisms are assembled to the output

$$0 \longleftarrow G_0 \xleftarrow{\begin{pmatrix} f_1^0 & p_0^1 \end{pmatrix}} G_1 \oplus R_0 \xleftarrow{\begin{pmatrix} f_2^0 & p_1^1 \\ h_2^0 & f_1^1 \end{pmatrix}} G_2 \oplus R_1 \xleftarrow{\begin{pmatrix} f_3^0 & p_2^1 \\ h_3^0 & f_2^1 \end{pmatrix}} G_3 \oplus R_2 \longleftarrow \cdots \tag{15}$$

which, as we will show now, is a free resolution of $C_\bullet$. Diagram (14) is commutative, that is, each square commutes and the maps h_i^0 satisfy

$$f_i^0 \circ f_{i+1}^0 = p_{i-1}^1 \circ h_{i+1}^0 \quad \text{and} \quad h_{i+1}^0 \circ p_{i+1}^1 = f_i^1 \circ f_{i+1}^1. \tag{16}$$

Here we note that the second property in (16) follows from the fact that

$$\begin{aligned} p_{i-1}^1 \circ (h_{i+1}^0 \circ p_{i+1}^1 + f_i^1 \circ f_{i+1}^1) &= f_i^0 \circ f_{i+1}^0 \circ p_{i+1}^1 + f_i^0 \circ p_i^1 \circ f_{i+1}^1 \\ &= f_i^0 \circ f_{i+1}^0 \circ p_{i+1}^1 + f_i^0 \circ f_{i+1}^0 \circ p_{i+1}^1 = 0 \end{aligned}$$

and p_{i-1}^1 is a monomorphism. Each horizontal sequence is exact, as $(G_i \xleftarrow{p_i^1} R_i, \alpha_i)$ is a free resolution of C_i.

Chain complex property. We show that the sequence in Diagram (15) is a chain complex. Indeed, it holds that

$$\begin{pmatrix} f_i^0 & p_{i-1}^1 \\ h_i^0 & f_{i-1}^1 \end{pmatrix} \circ \begin{pmatrix} f_{i+1}^0 & p_i^1 \\ h_{i+1}^0 & f_i^1 \end{pmatrix} = \begin{pmatrix} f_i^0 \circ f_{i+1}^0 + p_{i-1}^1 \circ h_{i+1}^0 & f_i^0 \circ p_i^1 + p_{i-1}^1 \circ f_i^1 \\ h_i^0 \circ f_{i+1}^0 + f_{i-1}^1 \circ h_{i+1}^0 & h_i^0 \circ p_i^1 + f_{i-1}^1 \circ f_i^1 \end{pmatrix} = 0$$

The upper right entry is zero because (f_i^0, f_i^1) is a chain map. The upper left and lower right entries are zero due to Equation (16). The postcomposition of the lower left entry with p_{i-2}^1 is zero by Equation (16). Thus, by exactness of the rows in Diagram (14), $\text{im}\,(h_i^0 \circ f_{i+1}^0 + f_{i-1}^1 \circ h_{i+1}^0) \subseteq \ker p_{i-2}^1 = 0$.

Quasi-isomorphism. The upper row of the following diagram is our output chain complex.

$$\begin{array}{ccccccccccccc} 0 & \longleftarrow & G_0 & \xleftarrow{\begin{pmatrix} f_1^0 & p_0^1 \end{pmatrix}} & G_1 \oplus R_0 & \xleftarrow{\begin{pmatrix} f_2^0 & p_1^1 \\ h_2^0 & f_1^1 \end{pmatrix}} & G_2 \oplus R_1 & \xleftarrow{\begin{pmatrix} f_3^0 & p_2^1 \\ h_3^0 & f_2^1 \end{pmatrix}} & G_3 \oplus R_2 & \longleftarrow & \cdots \\ & & \downarrow{\scriptstyle \alpha_0} & & \downarrow{\scriptstyle \begin{pmatrix} \alpha_1 & 0 \end{pmatrix}} & & \downarrow{\scriptstyle \begin{pmatrix} \alpha_2 & 0 \end{pmatrix}} & & \downarrow{\scriptstyle \begin{pmatrix} \alpha_3 & 0 \end{pmatrix}} & & \\ 0 & \longleftarrow & C_0 & \xleftarrow{\partial_1} & C_1 & \xleftarrow{\partial_2} & C_2 & \xleftarrow{\partial_3} & C_3 & \longleftarrow & \cdots. \end{array} \tag{17}$$

We show that $\alpha : \begin{pmatrix} \alpha_0 \end{pmatrix}, \begin{pmatrix} \alpha_1 & 0 \end{pmatrix}, \begin{pmatrix} \alpha_2 & 0 \end{pmatrix}, \ldots$ is a quasi-isomorphism between the complex in (15) and $C_\bullet$. For that reason, we show that the mapping cone $\text{cone}(\alpha)$ of the vertical maps in Diagram (17) is an acyclic complex which in turn implies that α is a quasi isomorphism (see Corollary 10.41 in [24]). $\text{cone}(\alpha)$ is the chain complex

$$0 \longleftarrow C_0 \xleftarrow{\begin{pmatrix} \partial_1 & \alpha_0 \end{pmatrix}} C_1 \oplus G_0 \xleftarrow{\begin{pmatrix} \partial_2 & \alpha_1 & 0 \\ 0 & f_1^0 & p_0^1 \end{pmatrix}} C_2 \oplus G_1 \oplus R_0 \xleftarrow{\begin{pmatrix} \partial_3 & \alpha_2 & 0 \\ 0 & f_2^0 & p_1^1 \\ 0 & h_2^0 & f_1^1 \end{pmatrix}} C_3 \oplus G_2 \oplus R_1 \longleftarrow \cdots \tag{18}$$

which is acyclic, if every cycle $(x, y, z) \in C_{i+1} \oplus G_i \oplus R_i$ is a boundary. Such a cycle fulfills

$$\begin{pmatrix} \partial_{i+1} & \alpha_i & 0 \\ 0 & f_i^0 & p_{i-1}^1 \\ 0 & h_i^0 & f_{i-1}^1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} \partial_{i+1}(x) + \alpha_i(y) \\ f_i^0(y) + p_{i-1}^1(z) \\ h_i^0(y) + f_{i-1}^1(z) \end{pmatrix} = 0. \tag{19}$$

Since α_{i+1} is an epimorphism, there exists $a \in G_{i+1}$ such that $\alpha_{i+1}(a) = x$. To get a boundary, another summand $b \in R_i$ satisfying

$$f_{i+1}^0(a) + p_i^1(b) = y$$

is necessary. Note that $\partial_{i+1} \circ \alpha_{i+1}(a) = \alpha_i \circ f^0_{i+1}(a)$ by Diagram 14. Hence by Equation (19),

$$\alpha_i \circ f^0_{i+1}(a) + \alpha_i(y) = \alpha_i(f^0_{i+1}(a) + y) = 0$$

and thus $f^0_{i+1}(a) + y \in \ker \alpha_i = \text{im } p^1_i$ by exactness of row i in Diagram (14). Thus there exists a $b \in R_i$ such that

$$p^1_i(b) + f^0_{i+1}(a) = y.$$

It remains to show that

$$h^0_{i+1}(a) + f^1_i(b) = z \tag{20}$$

in order to verify that $(0, a, b) \in C_{i+2} \oplus G_{i+1} \oplus R_i$ maps to (x, y, z) by the boundary operator in (18). Applying p^1_{i-1} to Equation (20) yields

$$p^1_{i-1}(h^0_{i+1}(a)+f^1_i(b)+z) = f^0_i(f^0_{i+1}(a)+p^1_i(b))+p^1_{i-1}(z) = f^0_i(y)+p^1_{i-1}(z) = 0$$

where the first equality follows by the commutativity of Diagram (14) and the third one by Equation (19). Equation (20) now holds since p^1_{i-1} is a monomorphism. This finishes the proof of Theorem 2.

An algebraic remark. We have constructed all morphisms in Diagram (14) explicitly in Section 4. Their existence and properties only are guaranteed by the following Lemma 9 from homological algebra, and only require the existence of free resolutions of length 1 of each C_i.

Lemma 9. (i) For any morphism $f : M \to N$ of bipersistence modules, $F_\bullet \xrightarrow{\epsilon_M} M$ and $G_\bullet \xrightarrow{\epsilon_N} N$ free resolutions there exists a lift $L(f) : F_\bullet \to G_\bullet$ such that

$$\begin{array}{ccc} M & \xleftarrow{\epsilon_M} & F_0 \\ {\scriptstyle f}\downarrow & & \downarrow{\scriptstyle L(f)_0} \\ N & \xleftarrow[\epsilon_N]{} & G_0 \end{array}$$

commutes.

(ii) Any two lifts $L(f)$ and $L'(f)$ are homotopic. This means that there exists a chain homotopy, which is a collection of morphisms $(h_i : F_i \to G_{i+1})_{i\in\mathbb{N}}$ such that

$$L(f)_i - L'(f)_i = \partial^G_{i+1} \circ h_i + h_{i-1} \circ \partial^F_i \qquad \text{for all } i \geq 0.$$

Indeed, the morphisms $f_i^0 : G_i \to G_{i-1}$ and $f_i^1 : R_i \to R_{i-1}$ are lifts of the boundary maps ∂_i according to Lemma 9 and they further assemble to a chain map $f_i^\bullet = (f_i^0, f_i^1)$ between the chain complexes $G_i \xleftarrow{p_i^1} R_i$ and $G_{i-1} \xleftarrow{p_{i-1}^1} R_{i-1}$. The composition $f_{i-1}^\bullet \circ f_i^\bullet$ is then a chain map between $G_i \xleftarrow{p_i^1} R_i$ and $G_{i-2} \xleftarrow{p_{i-2}^1} R_{i-2}$ and more specifically a lift of $\partial_{i-1} \circ \partial_i = 0$. By Lemma 9, $f_{i-1}^\bullet \circ f_i^\bullet$ any two lifts are unique up to homotopy. As the zero map also lifts $\partial_{i-1} \circ \partial_i$, the map $f_{i-1}^\bullet \circ f_i^\bullet$ is homotopic to zero. Thus, we identify this homotopy with its only constituting map h_i^0.

A.2 Complexity

Multiplying sparse matrices. Given two matrices $A \in \mathbb{Z}_2^{n\times p}$ and $B \in \mathbb{Z}_2^{p\times m}$ stored in sparse matrix format, i.e. as a list of columns represented by the row-indices of non-zero entries. Assume that the columns of A and B have length at most l and q, respectively. We compute the i-th column of $A \cdot B = (AB_1, \ldots, AB_m)$ by summing the columns of A indexed by the i-th column of B. This can be done by creating an accumulator array with zero entries of size n representing one column of $A \cdot B$. We can then compute the sum of the columns of A indexed by B_i by accumulating the non-zero entries in this array via bit flips. This can be done by going over all columns of A indexed by B_i once while remembering which bits are touched. After the column AB_i is finished we can clear the array. This can be done in $O(lq)$ time. Thus overall we can compute the product in $O(mql)$ time with an additional overhead of $O(n)$ for creating the accumulator array.

Proposition 10. Given a chain complex $C_\bullet$ induced by a k-critical bifiltration of size n. The time complexity of the path algorithm is linear in the description size of its output, being $O(nk)$ in the worst case.

Proof. The input is induced by a k-critical bifiltration $\mathcal{K}$ of constant dimension d with description size n, that is, the size of the input $C_\bullet$

is the cardinality of $\bigsqcup_{\sigma\in\mathcal{K}}\mathcal{G}(\sigma)$. We assume that for each simplex $\sigma\in\mathcal{K}$, $\mathcal{G}(\sigma):=n^\sigma\leq k$. The size of the output is determined by all non-zero entries of the matrices $[f_i^0],[p_i^1],[f_i^1]$ and $[h_i^0]$. Step 1 involves a simple iteration through all grades $\mathcal{G}(\sigma)$ and can thus be computed in $O(n)$ time. Note that all $[p_i^1]$ have size $O(n)$. In Step 2, we need to find generators $g_{\tau_0}^{x_0},\dots,g_{\tau_\ell}^{x_\ell}$ for each generator g_σ^x such that $x_j\leq x$. Here $\tau_j\in\partial\ \sigma$. Since there are at most k generators for each τ_j and the dimension is at most d, this can be done in $O(nd\log k)$ using binary search. All $[f_i^0]$ have size $O(nd)$. In Step 3, we first compute the matrix products $f_{i-1}^0\circ f_i^0$ and $f_i^0\circ p_i^1$. By construction, each column of f_i^0 has exactly $i+1\leq d+1$ non-zero entries, while each column of p_i^1 has exactly two non-zero entries. Because each involved matrix has $O(n)$ columns, these products of sparse matrices can be computed in $O(nd^2)$ time. The $O(d^2)$ non-zero entries of the columns of $f_{i-1}^0\circ f_i^0$ and $f_i^0\circ p_i^1$ consist of pairs of generators, which get connected by paths. Each such path can be found in $O(k)$ time, which results in the $[h_i^0]$ and $[f_i^1]$ matrices to have columns of size $O(d^2k)$, and all $[f_i^1]$ and $[h_i^0]$ having size $O(nd^2k)$ in total. This gives an overall size and time complexity of $O(nd^2k)=O(nk)$. □

A.3 Minimal resolutions of non-minimal description size

We consider the wheel example in Figure 8. It consists of an outer cycle of ℓ 1-critical vertices and edges, a central ℓ-critical vertex m, ℓ evenly and oddly labeled 1-critical edges that connect the vertices on the cycle to m and ℓ 1-critical faces. Because the only multi-critical simplex is the vertex m, it is the only one that in-

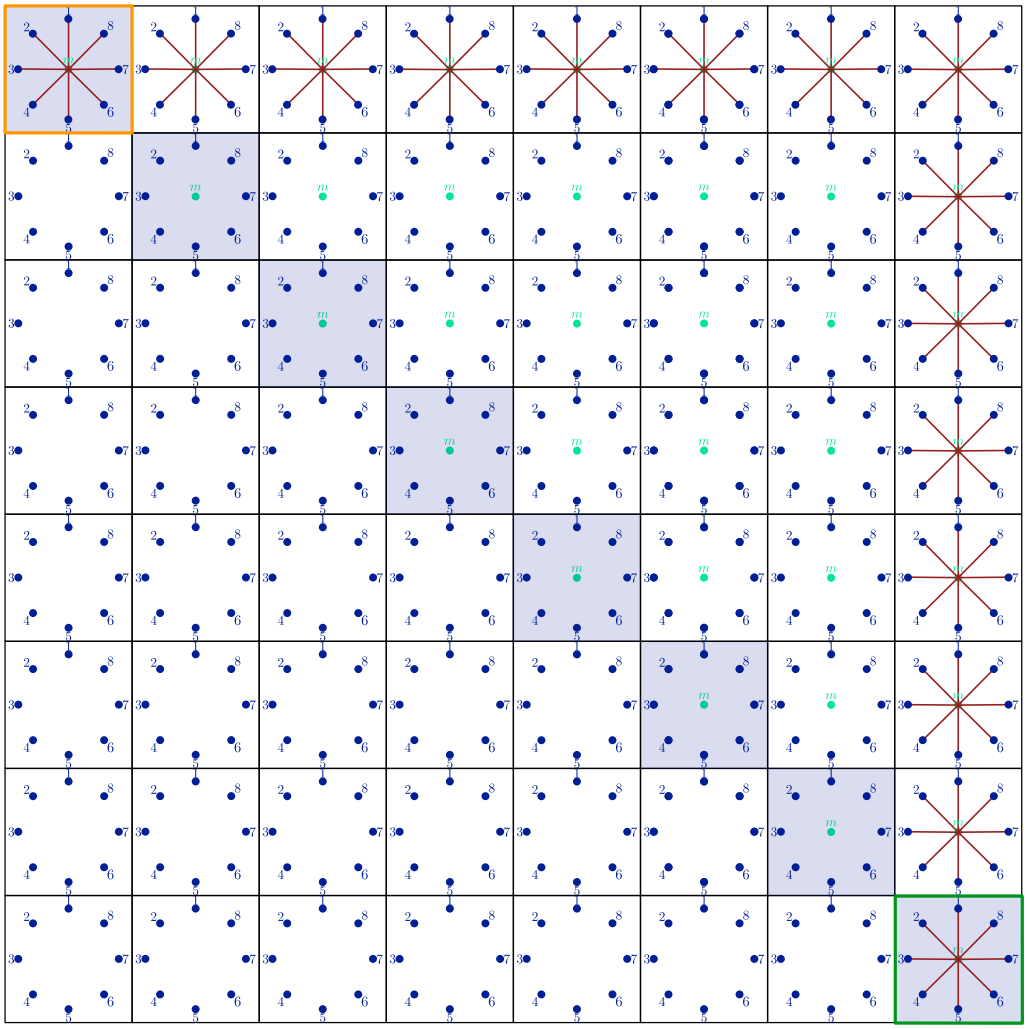
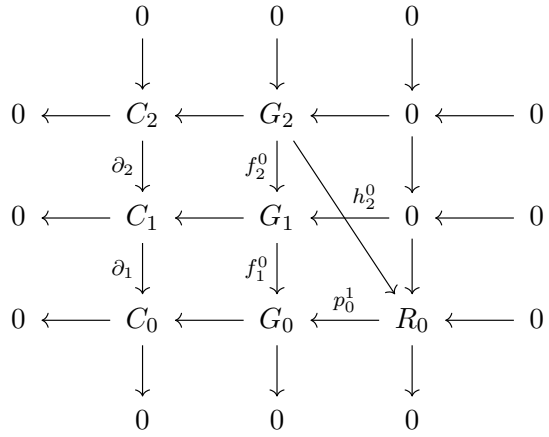


Figure 15: The star-bifiltration makes the matrix $[f_1^1]$ dense. It consists of $\ell+1$ 0-simplices and ℓ 1-simplices. The vertex m is ℓ-critical, entering the bifiltration along the blue staircase, while each 1-simplex is 2-critical, coming in at the orange and green grade. The map $f_1^0 \circ p_1^1$ maps each relation r_e of an edge e to a generator of m with yellow grade and a generator of m with green grade. These generators of m are connected by a path of length $\ell-1$ which constitute the column of $[f_1^1]$ indexed by r_e.

duces relations. Thus, we obtain the following diagram:



(21)

The map p_0^1 just sends each relation of m to its generators. The maps f_1^0 and f_2^0 are constructed by mapping each edge and face generator to a vertex and edge generator of its boundary, respectively. The only generators where there could be choices are the generators of the edges that have m in its boundary. But the edges and copies of m are positioned in a way such that each edge can only be mapped to a single generator of m. Hence, there is no choice in the construction of these maps and the even and odd edges are mapped to the endpoints of the path formed by the generators and relations of m. Because each face has an even and an odd edge in its boundary and the even and odd edges are mapped to the generators corresponding to these endpoints, $f_1^0 \circ f_2^0(g_\sigma^x) = g_m^y + g_m^z$ for each face σ. This implies that $h_2^0(g_\sigma^x)$ has to be defined as the path of relations of length ℓ connecting g_m^y and g_m^z for each of the $\ell - 1$ faces. The output of the algorithm is the chain complex

$$0 \longleftarrow G_0 \xleftarrow{\begin{pmatrix} f_1^0 & p_0^1 \end{pmatrix}} G_1 \oplus R_0 \xleftarrow{\begin{pmatrix} f_2^0 \\ h_2^0 \end{pmatrix}} G_2 \longleftarrow 0. \tag{22}$$

We now modify the wheel example in Figure 8, by slightly shifting all the edges such that every edge strictly comes after every vertex, all the edges and relations enter in incomparable grades

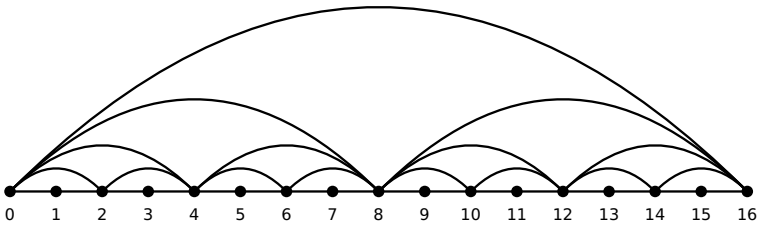


Figure 16: The log-path graph for $t = 4$.

and all the faces enter in incomparable grades without changing the relative comparability relations with the remaining simplices. This means that if two simplices (or copies thereof) are incomparable before this shift they are still incomparable after the shift. It is obvious that this can be done. After this modification every non-zero entry in the boundary matrices of (22), corresponds to a basis element b^x getting mapped to a basis element b^y such that $y < x$. This implies that the output chain complex is a minimal free resolution of $C_\bullet$ (see Definition 1.24 in [20]). Moreover, all basis elements corresponding to edges and relations or faces are incomparable. Therefore, the matrix $\binom{f_2^0}{h_2^0}$ has $O(\ell^2)$ non-zero entries and there is no possible basis transformation on G_2 or $G_1 \oplus R_0$ to reduce them. We conclude that the path-algorithm constructs a minimal free resolution from the modified wheel example that has a description size of $O(\ell^2)$. Compare this with Theorem 8 stating that it admits a free resolution of description size $O(\ell \log^2 \ell)$ as produced by the log-path algorithm.

B Proofs and details for the log-path algorithm

B.1 Finding shortest paths

In this section, we prove:

Lemma 11 (Log-path Lemma)**.** For any $g_\sigma^{x_j}$, $g_\sigma^{x_\ell}$ in G_σ, there exist $r_\sigma^{y_1}, \dots, r_\sigma^{y_m}$ in R_σ with $m = O(\log n_\sigma)$ such that $p_\sigma^1(r_\sigma^{y_1} + \dots + r_\sigma^{y_m}) = g_\sigma^{x_j} + g_\sigma^{x_\ell}$. Moreover, the elements $r_\sigma^{y_1}, \dots, r_\sigma^{y_m}$ can be computed in $O(\log n_\sigma)$ time.

To this end, we treat the 1-skeleton of the log-path resolution $\mathcal{L}_\sigma$ on a purely graph-theoretic level. To do so, we identify all generators as vertices labeled by numbers and define the construction as follows: We start with a path graph of length $m = 2^t$ and introduce $m - 1$ additional edges, functioning as shortcuts, such that any two vertices can be connected by a monotone path of length $O(\log(m)) = O(t)$. As illustrated in Figure 16, we first add shortcuts of length two for every other vertex. Then we add shortcuts of length four at every fourth vertex, shortcuts of length eight at every eighth vertex and so on. If m is not a power of two, we only add those edges that do not overshoot the last vertex of the path. Formally, for $t \in \mathbb{N}_0$, define the (undirected) graph $L = (V, E)$, where

$$V = \{0, \dots, 2^t\}$$
$$E = \{(x, y) | \exists r \in \mathbb{N}_0 \colon 2^r | x \text{ and } y = x + 2^r\}.$$

Note that $|V| = 2^t + 1$ and $|E| = \sum_{i=0}^{t} \frac{2^t}{2^i} = 2^t \sum_{i=0}^{t} 2^{-i} = 2^t(2 - 2^{-t}) = 2^{t+1} - 1$. Let $x, y \in L$ such that $x < y$, we call a path $x = z_0, z_1, \dots, z_l = y$ in L from x to y monotone if $z_i < z_{i+1}$ for all $0 \leq i < l$. We can construct a shortest monotone path between x and y in the following way. **Shortest monotone path algorithm:** Start at $x = z_0$ and choose a maximal r_0 with the property that $2^{r_0} | z_0$ and $z_0 + 2^{r_0} \leq z_l$. Set $z_1 \coloneqq z_0 + 2^{r_0}$. In other words, take the biggest possible step towards z_l that does not overshoot it. Then repeat this step by setting $z_{i+1} \coloneqq z_i + 2^{r_i}$ with r_i maximal such that $2^{r_i} | z_i$ and $z_i + 2^{r_i} \leq z_l$ until $z_l = y$ is reached. The proof of Lemma 1 now follows from the following Lemma.

Lemma 12. Given $x < y \in L$, the algorithm above computes the unique shortest monotone path $x = z_0, \ldots, z_l = y$ of length $O(t)$ in $O(t)$ time.

The following technical lemma is not only essential for the proof of Lemma 12, it also guarantees the planarity of L. Indeed, we can always draw L as in Figure 16. If (x, z) is an edge in L and (y, w) an edge such that $x < y < z$, then $x < y < w < z$. In words, all edges starting below the edge (x, z) stay below (x, z). In particular, L is a planar graph.

Lemma 13. If (x, z) is an edge in L and $x < y < z \in L$ such that $2^r | y$, then $y + 2^r \leq z$.

Proof. If (x, z) is an edge in L, there exists q such that $2^q | x$ and $z = x + 2^q$. Since $x < y < z$, we have $0 < y - x < z - x = 2^q$. If $r \geq q$, then $2^q | y$ and, thus, $2^q | (y - x)$. But this would imply $y - x = 2^q a$ with $a \geq 1$ and $2^q a < 2^q$, which is a contradiction. Hence, we have $r < q$ which implies $2^r | x$ and $2^r | (y - x)$. This allows us to write $y - x = 2^r b$ with an integer $b \geq 1$. Since $y - x < z - x$, we get $2^r b < 2^q$ and $b < 2^{q-r}$ or $b + 1 \leq 2^{q-r}$. Therefore,

$$y + 2^r = x + (y - x) + 2^r = x + 2^r b + 2^r = x + 2^r (b + 1) \leq x + 2^r 2^{q-r} = x + 2^q = z.$$

□

Proof of Lemma 12. By construction, $z_1 = x + 2^{r_0}$, where r_0 is maximal with the property $2^{r_0} | x$ and $x + 2^{r_0} \leq y$. This is the largest possible step towards y. Any monotone path from x to y that does not have (x, z_1) as its first edge must use a shorter first edge (x, a) with $x < a < z_1$. But by Lemma 13, any edge starting at a ends at or before z_1. Thus, every monotone path from x to y has to visit the vertex z_1. Since we could replace the part of any monotone path going from x to z_1 by the single edge (x, z_1) any shortest monotone path has to use (x, z_1) as its first edge. By applying the same argument to the path from z_1 and y, we obtain that the construction above yields the unique monotone shortest path from x to y. Let $x = z_0, \ldots, z_l = y$ be the shortest monotone path as constructed as above. Let r_i be the integer such that $2^{r_i} | z_i$ and $z_{i+1} = z_i + 2^{r_i}$. If r_i is maximal with the property that $2^{r_i} | z_i$, then $z_i = 2^{r_i} a_i$ with a_i odd. Thus, $z_{i+1} = z_i + 2^{r_i} = 2^{r_i} a_i + 2^{r_i} = 2^{r_i}(a_i + 1)$ with $(a_i + 1)$ even and $r_{i+1} > r_i$ as long as $z_{i+1} + 2^{r_{i+1}} \leq y$. Therefore, we take increasingly larger steps until we either reach y or reach

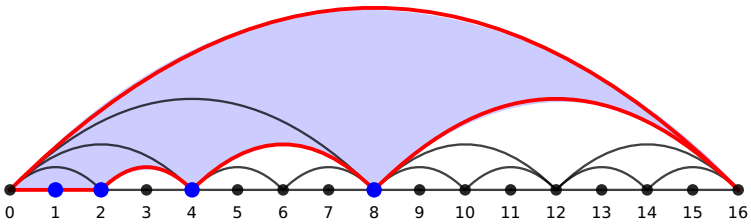


Figure 17: A simple red cycle in L for $t = 4$, together with the filling triangles in blue corresponding to the inner vertices.

a point where the biggest possible step would overshoot y. Since $y - x \leq 2^t$, we can take at most $t - 1$ of these increasing steps. If we reach a point z_j where $z_j + 2^{r_j} \leq y$ but $z_j = 2^{r_j} b_j$ with b_j even, then $z_{j+1} = z_j + 2^{r_j} = 2^{r_j}(b_j + 1)$ with $(b_j + 1)$ odd. If $r_{j+1} = r_j$, then $z_{j+2} = z_{j+1} + 2^{r_j} = z_j + 2^{r_j+1}$ but this overshoots y by construction of r_j. Thus, $r_{j+1} < r_j$. By repeating this argument we have to take steps of decreasingly smaller size. Since r_j is at most $t-1$, except for the case of $x = 0$ and $y = 2^t$ where the shortest path is of length one, we can take at most $t - 1$ such decreasing steps. Therefore, overall the shortest path is of length at most $2(t - 1)$. This bound is sharp as it it realized by the shortest monotone path from 1 to $2^t - 1$. The argument above also shows that the algorithm does not have to check all t possible values to find the maximal r_i such that $2^{r_i} | z_i$ and $z_i + 2^{r_i} \leq y$. We only have to increase r_i up to the point where we would first overshoot y and from there on we only decrease it. Hence, we only have to scan through all possible values of t at most twice. Since the constructed path has length smaller than $2(t - 1)$, this algorithm takes at most $2(t - 1) + 2t$ steps. □

B.2 Filling Cycles

In this section we prove:

Lemma 14. Let $r_\sigma^{y_1}, \dots, r_\sigma^{y_l}$ be a cycle in R_σ. Then there exists a unique chain $s_\sigma^{z_1} + \cdots + s_\sigma^{z_q}$ of $q \leq l-2$ elements in S_σ such that $p_\sigma^2(s_\sigma^{z_1} + \cdots + s_\sigma^{z_q}) = r_\sigma^{y_1} + \cdots + r_\sigma^{y_l}$. Moreover, this chain can be computed in $O(l) = O(\log k)$ time.

The log-path resolution $\mathcal{L}_\sigma$ has the structure of L viewed as a simplicial complex where all the inner triangles are filled. The triangles correspond to the syzygies of $\mathcal{L}_\sigma$. In other words, we have to solve the task of finding triangles that fill a given cycle in L, as illustrated in Figure 17. In this Section, we describe an efficient algorithm to solve this task. The Lemma then follows directly from Lemma 17. We first focus on simple cycles, i.e., cycles that have no repeating vertices (self intersections). Moreover, we assume that the cycles are fully canceled over $\mathbb{Z}_2$, i.e., there are no repeating edges. First we observe that we can identify every triangle with a vertex. By construction, every triangle consists of two edges of length 2^r and one edge of length 2^{r+1}. In other words, each triangle can be written as (a, b, c) where $a < b < c$, $b - a = 2^r$ and $c - b = 2^r$. We can identify each triangle with the vertex opposite to the longest edge, i.e., $(a, b, c) \sim b$. If (a, c) is the longest edge of a triangle, then $b = \frac{a+c}{2}$. The only vertices that are not matched with a triangle in this way are 0 and 2^t. Conversely, if $0 < x < 2^t$ is a vertex in L, and r is maximal such that $2^r | x$, then $(x - 2^r, x, x + 2^r)$ is a triangle in L. Thus, we also identify $x \sim (x - 2^r, x, x + 2^r)$ and obtain a bijection between the interior vertices on L and the triangles.

Lemma 15. Each simple cycle in L has a unique longest edge.

Proof. Let $x_0, \dots, x_l$ be the vertices of a simple cycle in L. It is clear that there exists a longest edge. W.l.o.g. assume that (x_0, x_l) is a longest edge and $x_0 < x_l$. This edge needs to have length $x_l - x_0 \geq 2$, as otherwise $x_0, \dots, x_l$ would not be a cycle. Each edge of length greater than one cuts the graph into two parts, as depicted in Figure 17. We can visualize it as a part lying under an arc and a part over an arc. Since the cycle is simple and the graph planar, we can not repeat a vertex and the remaining cycle has to completely lie on one of the two sides. If this cycle lies under (x_0, x_l), then that same edge needs to be its longest edge by Lemma 13. If this cycle lies over (x_0, x_l), then the only way to get back to x_0 from x_l is to take a longer edge over (x_0, x_l) which

contradicts that (x_0, x_l) is the longest edge. Therefore, (x_0, x_l) is the unique longest edge of the simple cycle. □

Lemma 16. If $x_0, \dots, x_l$ are the vertices on a simple cycle in L with longest edge (x_0, x_l), then we can fill the cycle with the triangles corresponding to $x_1, \dots, x_{l-1}$. In other words,

$$\partial \left[\sum_{i=1}^{l-1} (x_i - 2^{r_i}, x_i, x_i + 2^{r_i}) \right] = (x_0, x_l) + \sum_{i=0}^{l-1} (x_i, x_{i+1}).$$

Proof. We have already observed (in the proof of Lemma 15) that the path $x_0, \dots, x_l$ has to lie under the longest edge (x_0, x_l). The point $\frac{x_0+x_l}{2}$ also lies in the part of the graph lying under (x_0, x_l) and further cuts it into two components (the only edge of the induced subgraph going over it is (x_0, x_l)). Thus, the cycle has to visit the point $\frac{x_0+x_l}{2} = x_i$ for some $0 < i < l$. By adding the edges (x_0, x_i) and (x_i, x_l), we obtain two simple cycles $x_0, \dots, x_i$ and $x_i, \dots, x_l$ which might be trivial (consisting of two edges (x_0, x_i) or (x_i, x_l)). In each of these simple cycles (x_0, x_i) and (x_i, x_l) are the longest edge, because they are the longest edges in the respective induced subgraphs. If Δ_1 and Δ_2 are collections of triangles that fill the cycles $x_0, \dots, x_i$ and $x_i, \dots, x_l$, respectively, then $\Delta_1 + \Delta_2 + (x_0, x_i, x_l)$ fills the cycle $x_0, \dots, x_l$. Therefore, we can repeat the same argument for non-trivial cycles until they become trivial. In this way, we obtain that $\sum_{i=1}^{l-1} (x_i - 2^{r_i}, x_i, x_i + 2^{r_i})$ fills $x_0, \dots, x_l$. □

By Lemma 16, we can fill a simple cycle of length l in $O(l)$ time. If we are given an arbitrary fully canceled cycle $x_0, \dots, x_l$, we can decompose it into simple cycles and then apply the same argument again. If we are only given an unordered cycle graph, we can compute an ordered closed walk in linear time by a simple greedy traversal that marks visited edges (sometimes referred to as Hierholzer's algorithm). Hence, we assume the a closed walk as an input and use the following algorithm to decompose it into simple cycles.
Input: a closed walk $x_0, \dots, x_l$. Maintain a stack S of vertices and a map $\mathrm{pos} \colon V \to \mathbb{N}_0$ that stores the index of a vertex in S.

1. Initialize $S \leftarrow [\,]$, $\mathrm{pos}[-] = -1$, and an empty list $\mathcal{C}$ of cycles.
2. For $i = 0, 1, \dots, l-1$:

- If $\text{pos}[x_i] = -1$: push x_i onto S and set $\text{pos}[x_i] \leftarrow |S| - 1$.
- Else (a repeat): let $j = \text{pos}[x_i]$. Append the cycle $(S[j], S[j+1], \ldots, S[|S|-1])$ to $\mathcal{C}$. Then pop $S[j+1], \ldots, S[|S|-1]$ off the stack and set $\text{pos}[S[r]] = -1$ for $j < r \leq |S| - 1$ (keep $S[j] = x_i$).

3. Return $\mathcal{C}$.

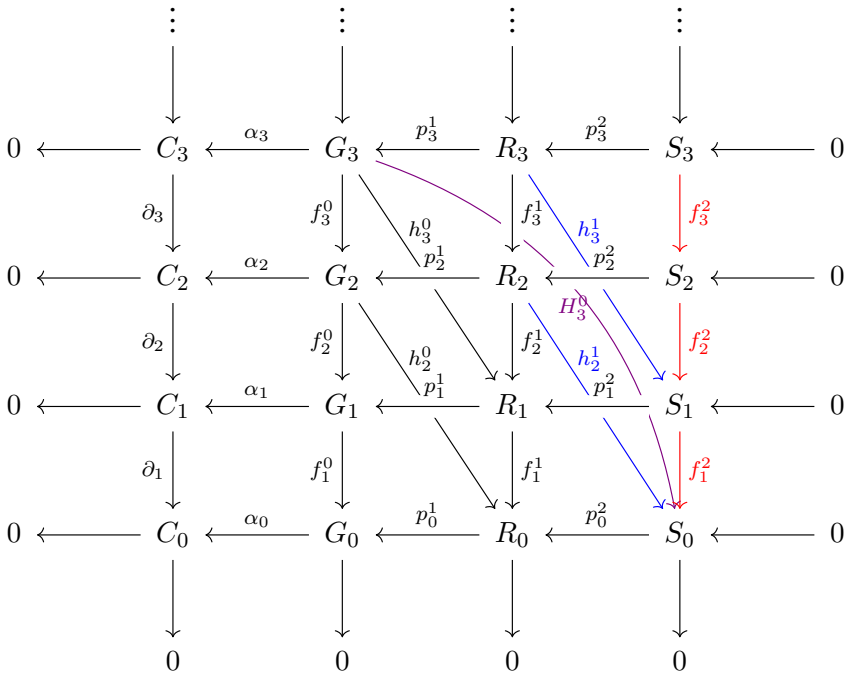
The algorithm goes over the closed walk and remembers which vertices are already visited. As long as we do not hit an already visited vertex all vertices on the current stack are distinct. If the walk hits an already visited vertex the first time, then the part of the walk since the repeated vertex forms a simple cycle. After removing this simple cycle from the stack the remaining vertices form a vertex distinct walk again. We proceed in this way until all input vertices are processed. After termination $\mathcal{C}$ contains a decomposition of the input into simple cycles. The initialization of pos takes $O(|V|)$ time. We process the input sequence by a single pass. Every vertex in the input sequence is put onto the stack at most once and removed from the stack at most once. The lookup in pos takes $O(1)$. Therefore, the time complexity is $O(l)$.

Lemma 17. Given a list of edges $e_0, \ldots, e_l$ forming a cycle in L, we can find filling triangles in $O(l)$ time.

Proof. We now combine all the previous arguments in the Section. Let $\{e_0, \ldots, e_l\}$ be a potentially unordered list of edges that forms a cycle in L. Assuming a global map from edges to their boundary vertices, we can build a graph data structure, like an adjacency list, representing the cycle graph C in linear time. We can then compute an Euler tour in C in linear time. Given this Euler tour, we can decompose it into simple cycles in linear time, using the algorithm discussed above. For each simple cycle, we can directly read off the filling triangles in linear time by Lemma 16. For that we need to know the longest edge but we can compute this during the cycle decomposition without overhead. Note that for the identification of vertices and triangles we do not have to check for each vertex what is the biggest power of two dividing it. We can just compute this identification while building the graph. Each triangle also corresponds to its longest edge (a, b) and then the corresponding vertex is $\frac{a+b}{2}$. □

B.3 Proof of quasi-isomorphism

This section is devoted to the proof of Theorem 7. We restate Diagram 10 for the convenience of the reader.



(23)

It is constructed from the input chain complex $C_\bullet$, induced by a simplicial bifiltration, in the following way: The i-th row is the sum of the log-path resolutions $\mathcal{L}_\sigma$ over all i-simplices and is, thus, a free resolution of C_i. The maps $f_i^\bullet$ are constructed such that the squares commute and are lifts of the boundary maps ∂_i according to Lemma 9. Moreover, the maps h_i^0, h_i^1, H_i^0 in (23) are constructed such that they satisfy

$$\begin{aligned} p_{i-2}^1 \circ h_i^0 &= f_{i-1}^0 \circ f_i^0 \\ h_i^0 \circ p_i^1 + p_{i-2}^2 \circ h_i^1 &= f_{i-1}^1 \circ f_i^1 \\ h_i^1 \circ p_i^2 &= f_{i-1}^2 \circ f_i^2 \\ h_{i-1}^0 \circ f_i^0 + f_{i-2}^1 \circ h_i^0 &= p_{i-3}^2 \circ H_i^0. \end{aligned} \tag{24}$$

We note again that the property $h_i^1 \circ p_i^2 = f_{i-1}^2 \circ f_i^2$ follows from the fact that p_{i-2}^2 is a monomorphism, as in A.1. The construction of

these maps is based on the exactness of the rows in (23) and

Lemma 18. The following morphisms map to the kernel of p_i^1:

$$f_{i+1}^1 \circ p_{i+1}^2, \quad f_{i+1}^1 \circ f_{i+2}^1 + h_{i+2}^0 \circ p_{i+2}^1, \quad \text{and} \quad h_{i+2}^0 \circ f_{i+3}^0 + f_{i+1}^1 \circ h_{i+3}^0. \tag{11}$$

Proof. By commutativity of Diagram 23, it holds that

$$p_i^1 \circ (f_{i+1}^1 \circ p_{i+1}^2) = f_{i+1}^0 \circ p_{i+1}^1 \circ p_{i+1}^2 = 0$$

and

$$p_i^1 \circ (f_{i+1}^1 \circ f_{i+2}^1 + h_{i+2}^0 \circ p_{i+2}^1) = f_{i+1}^0 \circ f_{i+2}^0 \circ p_{i+2}^1 + f_{i+1}^0 \circ f_{i+2}^0 \circ p_{i+2}^1 = 0$$

and

$$\begin{aligned} p_i^1 \circ (h_{i+2}^0 \circ f_{i+3}^0 + f_{i+1}^1 \circ h_{i+3}^0) &= f_{i+1}^0 \circ f_{i+2}^0 \circ f_{i+3}^0 + f_{i+1}^0 \circ p_{i+1}^1 \circ h_{i+3}^0 \\ &= f_{i+1}^0 \circ f_{i+2}^0 \circ f_{i+3}^0 + f_{i+1}^0 \circ f_{i+2}^0 \circ f_{i+3}^0 = 0. \end{aligned}$$

□

Remark 19. On a high level, these maps can be understood in the following way. Again by Lemma 9, the composition $f_{i-1}^\bullet \circ f_i^\bullet$ lifts the zero morphism $\partial_{i-1} \circ \partial_i = 0$ and is thus homotopic to zero. Hence, there exists a chain homotopy $h_i^\bullet$ such that $p_{i-2}^\bullet \circ h_i^\bullet + h_i^\bullet \circ p_i^\bullet = f_{i-1}^\bullet \circ f_i^\bullet$. Similarly, the composition $f_{i-2}^\bullet \circ f_{i-1}^\bullet \circ f_i^\bullet$ lifts $\partial_{i-2} \circ \partial_{i-1} \circ \partial_i = 0$ and is therefore homotopic to zero. In this case, the compositions $f_{i-2}^\bullet \circ h_i^\bullet$ and $h_{i-1}^\bullet \circ f_i^\bullet$ constitute homotopies between zero and $f_{i-2}^\bullet \circ f_{i-1}^\bullet \circ f_i^\bullet$. Such homotopies are again unique up to a higher homotopy $H_i^\bullet$ such that $p_{i-3}^\bullet \circ H_i^\bullet + H_i^\bullet \circ p_i^\bullet = f_{i-2}^\bullet \circ h_i^\bullet + h_{i-1}^\bullet \circ f_i^\bullet$.

We now show that given Diagram 23, with the properties discussed above, we obtain the following:

Proposition 20. The upper row of (25) is a chain complex and the vertical maps form a morphism of chain complexes.

$$\begin{array}{ccccccccccccc} 0 & \longleftarrow & G_0 & \xleftarrow{\begin{pmatrix} f_1^0 & p_0^1 \end{pmatrix}} & G_1 \oplus R_0 & \xleftarrow{\begin{pmatrix} f_2^0 & p_1^1 & 0 \\ h_2^0 & f_1^1 & p_0^2 \end{pmatrix}} & G_2 \oplus R_1 \oplus S_0 & \xleftarrow{\begin{pmatrix} f_3^0 & p_2^1 & 0 \\ h_3^0 & f_2^1 & p_1^2 \\ H_3^0 & h_2^1 & f_1^2 \end{pmatrix}} & G_3 \oplus R_2 \oplus S_1 & \longleftarrow & \cdots \\ & & \downarrow{\scriptstyle \alpha_0} & & \downarrow{\scriptstyle \begin{pmatrix} \alpha_1 & 0 \end{pmatrix}} & & \downarrow{\scriptstyle \begin{pmatrix} \alpha_2 & 0 & 0 \end{pmatrix}} & & \downarrow{\scriptstyle \begin{pmatrix} \alpha_3 & 0 & 0 \end{pmatrix}} & & \\ 0 & \longleftarrow & C_0 & \xleftarrow{\partial_1} & C_1 & \xleftarrow{\partial_2} & C_2 & \xleftarrow{\partial_3} & C_3 & \longleftarrow & \cdots \end{array} \tag{25}$$

Proof. We take the composition of two consecutive differentials in (25):

$$\begin{pmatrix} f_i^0 & p_{i-1}^1 & 0 \\ h_i^0 & f_{i-1}^1 & p_{i-2}^2 \\ H_i^0 & h_{i-1}^1 & f_{i-2}^2 \end{pmatrix} \circ \begin{pmatrix} f_{i+1}^0 & p_i^1 & 0 \\ h_{i+1}^0 & f_i^1 & p_{i-1}^2 \\ H_{i+1}^0 & h_i^1 & f_{i-1}^2 \end{pmatrix} =$$
$$\begin{pmatrix} f_i^0 \circ f_{i+1}^0 + p_{i-1}^1 \circ h_{i+1}^0 & f_i^0 \circ p_i^1 + p_{i-1}^1 \circ f_i^1 & p_{i-1}^1 \circ p_{i-1}^2 \\ h_i^0 \circ f_{i+1}^0 + f_{i-1}^1 \circ h_{i+1}^0 + p_{i-2}^2 \circ H_{i+1}^0 & h_i^0 \circ p_i^1 + f_{i-1}^1 \circ f_i^1 + p_{i-2}^2 \circ h_i^1 & f_{i-1}^1 \circ p_{i-1}^2 + p_{i-2}^2 \circ f_{i-1}^2 \\ H_i^0 \circ f_{i+1}^0 + h_{i-1}^1 \circ h_{i+1}^0 + f_{i-2}^2 \circ H_{i+1}^0 & H_i^0 \circ p_i^1 + h_{i-1}^1 \circ f_i^1 + f_{i-2}^2 \circ h_i^1 & h_{i-1}^1 \circ p_{i-1}^2 + f_{i-2}^2 \circ f_{i-1}^2 \end{pmatrix}.$$

All of the entries of the matrix representation of the composition are zero because of the exactness of the rows or commutativity in (23) or by the properties (24), except for $H_i^0 \circ f_{i+1}^0 + h_{i-1}^1 \circ h_{i+1}^0 + f_{i-2}^2 \circ H_{i+1}^0$. But for this entry we obtain:

$$\begin{aligned} & p_{i-3}^2 \circ \left[H_i^0 \circ f_{i+1}^0 + h_{i-1}^1 \circ h_{i+1}^0 + f_{i-2}^2 \circ H_{i+1}^0\right] \\ & = \left[h_{i-1}^0 \circ f_i^0 + f_{i-2}^1 \circ h_i^0\right] \circ f_{i+1}^0 + \left[f_{i-2}^1 \circ f_{i-1}^1 + h_{i-1}^0 \circ p_{i-1}^1\right] \circ h_{i+1}^0 + \left[f_{i-2}^1 \circ p_{i-2}^2\right] \circ H_{i+1}^0 \\ & = h_{i-1}^0 \circ f_i^0 \circ f_{i+1}^0 + f_{i-2}^1 \circ h_i^0 \circ f_{i+1}^0 + f_{i-2}^1 \circ f_{i-1}^1 \circ h_{i+1}^0 \\ & + h_{i-1}^0 \circ \left[f_i^0 \circ f_{i+1}^0\right] + f_{i-2}^1 \circ \left[h_i^0 \circ f_{i+1}^0 + f_{i-1}^1 \circ h_{i+1}^0\right] = 0. \end{aligned}$$

Since p_{i-3}^2 is a monomorphism, also this entry vanishes. Thus, (25) is indeed a chain complex. Similarly, we check that

$$\begin{pmatrix} \alpha_{i-1} & 0 & 0 \end{pmatrix} \circ \begin{pmatrix} f_i^0 & p_{i-1}^1 & 0 \\ h_i^0 & f_{i-1}^1 & p_{i-2}^2 \\ H_i^0 & h_{i-1}^1 & f_{i-2}^2 \end{pmatrix} = \begin{pmatrix} \alpha_{i-1} \circ f_i^0 & \alpha_{i-1} \circ p_{i-1}^1 & 0 \end{pmatrix} = \partial_i \circ \begin{pmatrix} \alpha_i & 0 & 0 \end{pmatrix}$$

and, thus, the vertical maps form a morphism of chain complexes. □

We are now ready to show that the morphism in (25) is a quasi-isomorphism and, therefore, the upper row of (25) is a free resolution of $C_\bullet$.

Theorem 7. The chain complex (13) is a free resolution of $C_\bullet$.

Proof of Theorem 7. We show that the mapping cone, cone(α):

$$0 \longleftarrow C_0 \xleftarrow{\begin{pmatrix} \partial_1 & \alpha_0 \end{pmatrix}} C_1 \oplus G_0 \xleftarrow{\begin{pmatrix} \partial_2 & \alpha_1 & 0 \\ 0 & f_1^0 & p_0^1 \end{pmatrix}} C_2 \oplus G_1 \oplus R_1 \xleftarrow{\begin{pmatrix} \partial_3 & \alpha_2 & 0 & 0 \\ 0 & f_2^0 & p_1^1 & 0 \\ 0 & h_2^0 & f_1^1 & p_0^2 \end{pmatrix}} C_3 \oplus G_2 \oplus R_1 \oplus S_0 \xleftarrow{\begin{pmatrix} \partial_4 & \alpha_3 & 0 & 0 \\ 0 & f_3^0 & p_2^1 & 0 \\ 0 & h_3^0 & f_2^1 & p_1^2 \\ 0 & H_3^0 & h_2^1 & f_1^2 \end{pmatrix}} C_4 \oplus G_3 \oplus R_2 \oplus S_1 \longleftarrow \cdots$$

of the chain morphism in (25) is acyclic. Assume

$$\begin{pmatrix} \partial_i & \alpha_{i-1} & 0 & 0 \\ 0 & f^0_{i-1} & p^1_{i-2} & 0 \\ 0 & h^0_{i-1} & f^1_{i-2} & p^2_{i-3} \\ 0 & H^0_{i-1} & h^1_{i-2} & f^2_{i-3} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} \partial_i(x_1) + \alpha_{i-1}(x_2) \\ f^0_{i-1}(x_2) + p^1_{i-2}(x_3) \\ h^0_{i-1}(x_2) + f^1_{i-2}(x_3) + p^2_{i-3}(x_4) \\ H^0_{i-1}(x_2) + h^1_{i-2}(x_3) + f^2_{i-3}(x_4) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

Since α_i is an epimorphism, there exists $y_1 \in G_i$ such that $\alpha_i(y_1) = x_1$. Then $\partial_i \circ \alpha_i(y_1) = \alpha_{i-1} \circ f^0_i(y_1) = \alpha_{i-1}(x_2)$ and, thus, $\alpha_{i-1}\big(f^0_i(y_1) + x_2\big) = 0$. By exactness, there exists $y_2 \in R_{i-1}$ such that $p^1_{i-1}(y_2) = f^0_i(y_1) + x_2$. Using this relation, we get:

$$f^0_{i-1}\big(x_2\big) = f^0_{i-1}\big(f^0_i(y_1) + p^1_{i-1}(y_2)\big) = p^1_{i-2} \circ h^0_i(y_1) + p^1_{i-2} \circ f^1_{i-1}(y_2) = p^1_{i-2}(x_3).$$

Hence, $p^1_{i-2}\big(h^0_i(y_1) + f^1_{i-1}(y_2) + x_3\big) = 0$ and, by exactness, there exists $y_3 \in S_{i-2}$ such that $p^2_{i-2}(y_3) = h^0_i(y_1) + f^1_{i-1}(y_2) + x_3$. Using this relation, we obtain

$$\begin{aligned} h^0_{i-1}(x_2) + f^1_{i-2}(x_3) &= h^1_{i-1}\big(f^0_i(y_1) + p^1_{i-1}(y_2)\big) + f^1_{i-2}\big(h^0_i(y_1) + f^1_{i-1}(y_2) + p^2_{i-2}(y_3)\big) \\ &= \left[h^1_{i-1} \circ f^0_i + f^1_{i-2} \circ h^0_i\right](y_1) + \left[h^1_{i-1} \circ p^1_{i-1} + f^1_{i-2} \circ f^1_{i-1}\right](y_2) + f^1_{i-2} \circ p^2_{i-2}(y_3) \\ &= p^2_{i-3} \circ H^0_i(y_1) + p^2_{i-3} \circ h^1_{i-1}(y_2) + p^2_{i-3} \circ f^2_{i-2}(y_3) = p^2_{i-3}(x_4). \end{aligned}$$

Hence, $p^2_{i-3}\big(H^0_i(y_1) + h^1_{i-1}(y_2) + f^2_{i-2}(y_3) + x_4\big) = 0$ and, since p^2_{i-3} is a monomorphism $x_4 = H^0_i(y_1) + h^1_{i-1}(y_2) + f^2_{i-2}(y_3)$. We conclude that

$$\begin{pmatrix} \partial_{i+1} & \alpha_i & 0 & 0 \\ 0 & f^0_i & p^1_{i-1} & 0 \\ 0 & h^0_i & f^1_{i-1} & p^2_{i-2} \\ 0 & H^0_i & h^1_{i-1} & f^2_{i-2} \end{pmatrix} \begin{pmatrix} 0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} \alpha_i(y_1) \\ f^0_i(y_1) + p^1_{i-1}(y_2) \\ h^0_i(y_1) + f^1_{i-1}(y_2) + p^2_{i-2}(y_3) \\ H^0_i(y_1) + h^1_{i-1}(y_2) + f^2_{i-2}(y_3) \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}$$

and, therefore, the mapping cone is exact. This implies that (25) is a quasi-isomorphism (see Corollary 10.41 in [24]). □

B.4 Complexity and correctness for the log-path algorithm

Theorem 8. A free resolution of the chain complex $C_\bullet$ induced by a k-critical bifiltration of description size n can be computed in $O(n \log^2 k)$ time and has $O(n \log^2 k)$ size.

Proof of Theorem 8. The correctness of the algorithm is a direct consequence of Theorem 7. In Step 1, we build $\mathcal{L}_\sigma$ for all input

simplices. This requires iterating through all simplex grades once and adding $O(n)$ relations and syzygies, which costs $O(n)$ time. In Step 2, we have to go over all generators g_σ^x and find boundary generators g_τ^y with $y \leq x$ for all facets τ of σ. Such a g_τ^y can be found in $O(\log n_\tau)$ time and, thus, the image of each g_σ^x can be determined in $O(d \log k)$ time. Hence, overall this step takes $O(nd \log k)$ time. For Step 3, we note that all involved matrices have $O(n)$ columns and, by construction, each column of f_i^0 has at most d non-zero entries while each column of p_i^1 has exactly two non-zero entries. Therefore, we can compute the sparse matrix product $f_i^0 \circ p_i^1$ and $f_{i-1}^0 \circ f_i^0$ in $O(nd^2)$ time. The columns of these products contain at most d^2 pairs of generators g_σ^x, $g_\sigma^{x'}$ which have to be connected by a shortest monotone path. Such a path can be found in $O(\log n_\sigma)$ time by Lemma 4. Thus, overall Step 3 takes $O(nd^2 \log k)$ time. For Step 4, we note that the matrix column sparsity of f_i^1 and h_i^0 is $O(d \log k)$ and $O(d^2 \log k)$, respectively, while in p_i^2 every column has exactly three non-zero entries. Hence, we can compute all sparse matrix products in $O(nd^3 \log^2 k)$ and the worst case column sparsity of the results is $O(d^3 \log^2 k)$. The resulting matrices contain cycles of length $O(d^3 \log^2 k)$ which can be filled by triangles in $O(d^3 \log^2 k)$ time by Lemma 5. Therefore, overall, Step 4 takes $O(nd^3 \log^2 k)$ time. Since we assume the dimension d is constant, we obtain an overall time complexity of $O(n \log^2 k)$. By the discussion of the column sparsity of the involved matrices, we also obtain that the description size of the output is $O(n \log^2 k)$. □

References

[1] Ángel Javier Alonso. A Sparse Multicover Bifiltration of Linear Size. In Oswin Aichholzer and Haitao Wang, editors, 41st International Symposium on Computational Geometry (SoCG 2025), volume 332 of Leibniz International Proceedings in Informatics (LIPIcs), pages 6:1–6:18, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[2] Anonymous. Compressed and approximate degree-rips bifiltrations. Submitted to SoCG 2026.

[3] Ulrich Bauer, Fabian Lenzen, and Michael Lesnick. Efficient Two-Parameter Persistence Computation via Cohomology. In Erin W. Chambers and Joachim Gudmundsson, editors, 39th International Symposium on Computational Geometry (SoCG 2023), volume 258 of Leibniz International Proceedings in Informatics (LIPIcs), pages 15:1–15:17, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[4] Dave Bayer and Bernd Sturmfels. Cellular resolutions of monomial modules. J. Reine Angew. Math., 502:123–140, 1998.

[5] Nello Blaser, Morten Brun, Odin Hoff Gardaa, and Lars M. Salbu. Core bifiltration. Journal of Applied and Computational Topology, 9(30), 2025.

[6] Andrew J. Blumberg and Michael Lesnick. Stability of 2-parameter persistent homology. Found. Comput. Math., 24(2):385–427, 2024.

[7] Wojciech Chachólski, Martina Scolamiero, and Francesco Vaccarino. Combinatorial presentation of multidimensional persistent homology. Journal of Pure and Applied Algebra, 221(5):1055–1075, 2017.

[8] Chao Chen and Michael Kerber. Persistent homology computation with a twist. In European Workshop on Computational Geometry (EuroCG), pages 197–200, 2011.

[9] David Coeurjolly, Jaques-Olivier Lachaud, Konstantinos Katrioplas, Sébastien Loriot, Ivan Paden, Mael Rouxel-Labbé, Hossam Saeed, Jane Tournois, and Ilker O. Yaz. Polygon mesh processing. In CGAL User and Reference Manual. CGAL Editorial Board, 6.0.1 edition, 2024.

[10] Tamal K. Dey and Florian Russold. Computing projective implicit representations from poset towers, 2025.

[11] Tamal K. Dey, Florian Russold, and Shreyas N. Samaga. Efficient Algorithms for Complexes of Persistence Modules with Applications. In Wolfgang Mulzer and Jeff M.

Phillips, editors, 40th International Symposium on Computational Geometry (SoCG 2024), volume 293 of Leibniz International Proceedings in Informatics (LIPIcs), pages 51:1–51:18, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[12] Kaspar Fischer, Bernd Gärtner, Thomas Herrmann, Michael Hoffmann, and Sven Schönherr. Bounding volumes. In CGAL User and Reference Manual. CGAL Editorial Board, 6.0.1 edition, 2024.

[13] Ulderico Fugacci and Michael Kerber. Chunk Reduction for Multi-Parameter Persistent Homology. In Gill Barequet and Yusu Wang, editors, 35th International Symposium on Computational Geometry (SoCG 2019), volume 129 of Leibniz International Proceedings in Informatics (LIPIcs), pages 37:1–37:14, Dagstuhl, Germany, 2019. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[14] Ulderico Fugacci, Michael Kerber, and Alexander Rolle. Compression for 2-parameter persistent homology. Computational Geometry, 109:101940, 2023.

[15] Michael Kerber. multi-chunk. `https://bitbucket.org/mkerber/multi_chunk/src/master/`, 2021. Accessed: 2025-12-02.

[16] Michael Kerber and Michael Lesnick. The scc2020 format. `https://bitbucket.org/mkerber/chain_complex_format/src/master/`, 2021. Accessed: 2025-12-02.

[17] Michael Kerber and Alexander Rolle. Fast minimal presentations of bi-graded persistence modules. In 2021 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX), pages 207–220. Software available at `https://bitbucket.org/mkerber/mpfree/src/master/`.

[18] Michael Lesnick and Matthew Wright. Interactive visualization of 2-d persistence modules, 2015.

[19] Michael Lesnick and Matthew Wright. Computing minimal presentations and bigraded betti numbers of 2-parameter

persistent homology. SIAM Journal on Applied Algebra and Geometry, 6(2):267–298, 2022.

[20] Ezra Miller and Bernd Sturmfels. Combinatorial Commutative Algebra. Graduate Texts in Mathematics, 227. Springer New York, New York, NY, 2005.

[21] Dmitriy Morozov and Luis Scoccola. Computing Betti tables and minimal presentations of zero-dimensional persistent homology. In Oswin Aichholzer and Haitao Wang, editors, 41st International Symposium on Computational Geometry, SoCG 2025, Kanazawa, Japan, June 23-27, 2025, volume 332 of LIPIcs, pages 69:1–69:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.

[22] Alexander Rolle. The Degree-Rips Complexes of an Annulus with Outliers. In Xavier Goaoc and Michael Kerber, editors, 38th International Symposium on Computational Geometry (SoCG 2022), volume 224 of Leibniz International Proceedings in Informatics (LIPIcs), pages 58:1–58:14, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[23] Alexander Rolle and Luis Scoccola. Stable and consistent density-based clustering via multiparameter persistence. Journal of Machine Learning Research, 25(258):1–74, 2024.

[24] Joseph J. Rotman. An Introduction to Homological Algebra. Springer New York, 2008.

[25] Gregory G. Smith and Mike Stillman. freeresolution(complex). `https://macaulay2.com/doc/Macaulay2/share/doc/Macaulay2/Complexes/html/index.html`, 2023. Accessed: 2025-12-02.

[26] Hans Tangelder and Andreas Fabri. dD spatial searching. In CGAL User and Reference Manual. CGAL Editorial Board, 6.0.1 edition, 2024.